The OpenNMS UI Studio Book: Topology, GraphML and Icons, Step by Step

Nov 2, 2026 · @ishtiaq

How to use this book

This book takes you from zero to changing how OpenNMS draws its topology map: first by hand on a development OpenNMS, then with the project's tools, and last on production. Read it once in order; afterwards it is your reference.

Who it is for. You run OpenNMS Horizon 33.1.8 in Podman for a Maven Wireless DAS (node types Orion, Helix, Stratus and Nimbus), and you want the map to show your devices with the right icons and your own look. Anyone who joins the work can follow it the same way.

What you will have at the end:

- a development OpenNMS that shows production's nodes and views;
- each DAS node type drawn with its own icon;
- your own look (backgrounds, frames, labels, links) as CSS, tried on the development OpenNMS;
- the same changes on production, with a backup and an undo;
- the word "Tools" instead of "Plugins" in the menu bar (the second, smaller module of the project).

The route. Times are rough estimates for a first pass.

Part	Chapters	You will	Time
Start	0	get to a known starting point: one folder, one version, a running lab	15 min
I Understand	1 to 3	learn what the project is, how OpenNMS draws its topology map, and what GraphML is	1 h of reading
II By hand	4 to 9	start your lab, then make nodes, a view, icons and CSS yourself, with podman and curl	4 to 6 h

Part	Chapters	You will	Time
III The tools	10 to 12	do the same with the topology kit and with OpenNMS UI Studio, and change the menu bar	2 to 3 h
IV Production	13	take your changes to production safely	1 h
Reference	Appendices A to E	look up a file, a command, a problem or a word	when needed

Why by hand first. The tools only automate the steps of Part II. Once you have done a step yourself, a tool's output reads as a short story you already know, and when a tool fails you can do the step without it.

How each chapter works. It starts with *Goal* and *You need*, explains the idea in a few paragraphs, then has exercises. Each exercise says what to *do*, what to *expect*, and how to *undo* it. A chapter ends with a *Checkpoint*: if a check fails, stop there and look it up in Appendix C before you go on.

Conventions.

Written as	Means
`dev$`	a terminal on the development machine (192.168.0.47, user `admin`); each chapter says in which folder
`kit$`	a terminal on the development machine in the kit folder, `~/opennms-fundamentals/opennms-ui-studio/kit` (from Chapter 4 on)
`prod$`	a terminal on production (192.168.0.36, user `maven-admin`)
`onms-dev-core`	the container of the development OpenNMS
`opennms`	the container of the production OpenNMS
`http://192.168.0.47:8980/opennms`	the development OpenNMS in your browser, user admin, password admin
`http://192.168.0.47:8990`	OpenNMS UI Studio in your browser (Part III)

Written as	Means
dev, *production*	the development OpenNMS, the production OpenNMS

Five golden rules.

1. Practise on dev. Production is touched in Chapter 13 only, with a backup.
2. One project folder: `~/opennms-fundamentals/opennms-ui-studio`. The older kit folders (`opennms-topology-kit-v4`, `-v5`, `-v6`) are retired.
3. Before you change a file by hand, keep a copy of it on the development machine: that copy is your way back.
4. A new svg file reaches the topology page only in a new browser tab; a changed file shows after a reload.
5. If you get lost, come back to the route above: every chapter says what it needs and what you should see at its end.

Chapter 0: A known starting point

Most of the confusion so far came from not knowing what was where: several kit folders, several versions, two OpenNMS. In 15 minutes this chapter gives you one folder at one version and a checklist that says you are ready.

Goal: the project folder at version 1.2.0, the tools it needs, and a clear answer to "is my lab running?". **You need:** a terminal on the development machine.

0.1 Three machines, two OpenNMS

Machine	Address, user	What runs there
Development machine	192.168.0.47, `admin` (CentOS Stream 9 VM)	the project folder, and the development OpenNMS: containers `onms-dev-core` (OpenNMS) and `onms-dev-db` (PostgreSQL) in a pod `onms-dev`
Production	192.168.0.36, `maven-admin`	the real OpenNMS: container `opennms`, next to `postgres-database` and `maven-nms`

Machine	Address, user	What runs there
Your computer	any	only a browser: it opens the web pages of both machines

Both OpenNMS run the same image, `docker.io/opennms/maven:33.1.8-1192-g05c63c46e8`. Dev is a copy you may break; production is not.

0.2 One project folder

The project is **OpenNMS UI Studio**. The **topology kit**, the command-line tools you used before, lives inside it as the folder `kit/`. Everything in this book happens in one folder:

```
~/opennms-fundamentals/opennms-ui-studio
```

Check what you have:

```
dev$ ls ~/opennms-fundamentals
dev$ grep '"version"' ~/opennms-fundamentals/opennms-ui-studio/package.json
```

The second line should print `"version": "1.2.0"`. If the folder is missing, or the version is older, take one of these two roads (Chapter 4.1 has the details):

You have	Do
nothing, or you want a clean start	unpack the full package `opennms-ui-studio-1.2.0.tar.gz` (sent with this book)
1.0.0	unpack the updates `opennms-ui-studio-1.1.0-update.tar.gz`, then `opennms-ui-studio-1.2.0-update.tar.gz`, over the folder
1.1.0	unpack `opennms-ui-studio-1.2.0-update.tar.gz` over the folder

The older kit folders (`~/opennms-fundamentals/opennms-fundamentals/opennms-topology-kit-v6` and earlier) are retired. Do not run their `dev.sh` any more: it would deploy that folder's own CSS to dev. The only thing you may still want from there is your CSS file, `styles/topology.css`; Chapter 11.3 imports it into the studio.

0.3 The tools the machine needs

```
dev$ podman --version      # Podman runs the development OpenNMS
dev$ python3 --version     # the kit's helpers are Python 3 scripts
dev$ curl --version | head -1
dev$ ssh maven-admin@192.168.0.36 podman ps --format '{{.Names}}'   # must
work without a password
dev$ node --version        # v20 or newer; needed only for the studio (Part
III)
dev$ git --version         # needed only for the studio's history
```

If `ssh` asks for a password, do this once and try again:

```
dev$ ssh-keygen -t ed25519            # press Enter at every question (skip
it if ~/.ssh/id_ed25519 exists)
dev$ ssh-copy-id maven-admin@192.168.0.36
```

Missing tools: `sudo dnf install -y podman python3 curl git`. Node.js: `sudo dnf module install -y nodejs:20/common`.

0.4 Is the lab running?

```
dev$ cd ~/opennms-fundamentals/opennms-ui-studio
dev$ kit/dev.sh status
```

- It lists `onms-dev-db` and `onms-dev-core` as *Up*, says OpenNMS is *ready* (or *web UI up*) and prints the topology address: your lab runs. Open that address in your browser.
- It says *There is no development OpenNMS yet*, or OpenNMS is not answering: Chapter 4.2 starts it.

Checkpoint

- [ ] `package.json` of `~/opennms-fundamentals/opennms-ui-studio` says 1.2.0
- [ ] `ssh maven-admin@192.168.0.36 podman ps` works without a password
- [ ] `kit/dev.sh status` answers, and you know whether the lab runs
- [ ] you will work in this one folder only, and leave the old kit folders alone

Part I: Understand

Chapter 1: The project in one picture

The project changes how OpenNMS draws its topology map (icons, colours, views) and its menu bar, safely: every change is tried on a development copy of OpenNMS, and only then copied to production, with a backup and an undo.

Goal: know the four pieces of the project, where each one runs, and the loop you will repeat. **You need:** nothing; this chapter is reading.

1.1 The four pieces

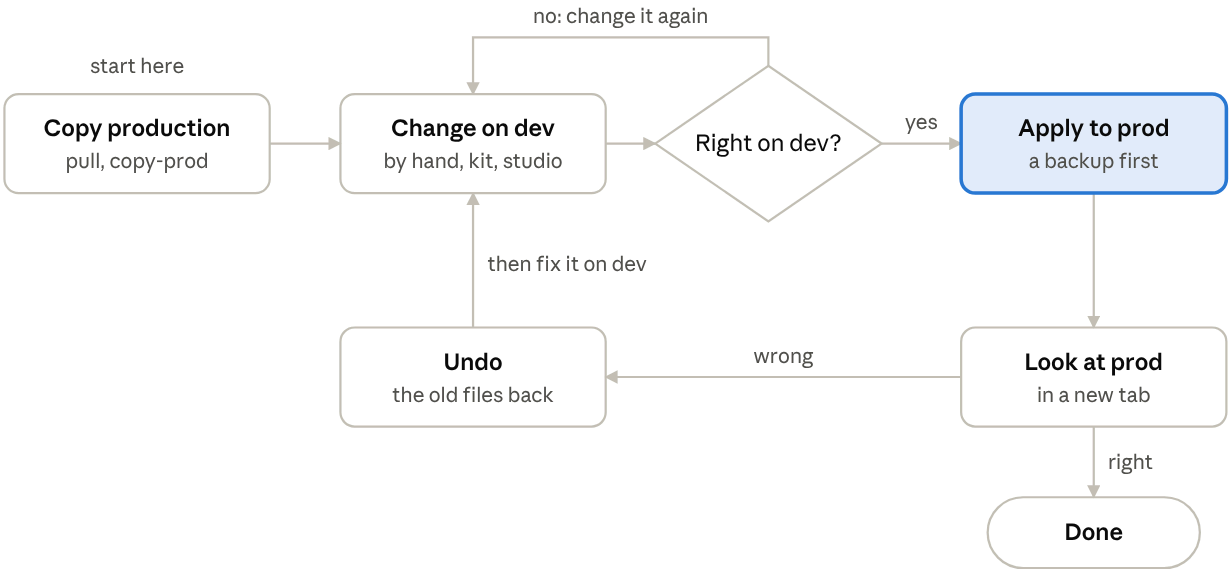
Piece	What it is	Where it runs	How you use it
Production OpenNMS	the real OpenNMS that monitors your DAS; its views, such as *Maven-abc*, come from your own software	container `opennms` on 192.168.0.36	its web UI; changed only in Chapter 13
Development OpenNMS (dev)	the same image with its own database: your lab	containers `onms-dev-core` and `onms-dev-db` on 192.168.0.47	`kit/dev.sh` starts it; you change it freely
The topology kit	shell and Python tools that copy files between production, dev and a folder, and check them	`kit/` in the project folder	commands such as `kit/topo.sh pull` (Chapter 10)
OpenNMS UI Studio	a web app: a CSS editor with a live preview of the map, the icon check, the menu bar words, Apply and Undo, history	`npm start` in the project folder, port 8990	your browser (Chapter 11)

The kit and the studio do the same things you will do by hand in Part II: read files out of a container, change them, put them back, and tell OpenNMS to read them again.

1.2 The loop you will repeat

1. **Bring production's state to dev**: its icons, mapping files and views (`kit/topo.sh pull`), and its nodes (`kit/dev.sh copy-prod`).
2. **Change something on dev**: by hand (Part II), with the kit, or in the studio.
3. **Look** at the topology page of dev in your browser. Not right? Back to step 2.
4. **Apply it to production**: the studio's *Apply* or `kit/topo.sh deploy prod`. Both keep a copy of every file they replace.
5. **Look** at production. Wrong? *Undo* puts the old files back.

Every change is made and checked on dev before it reaches production



the loop of the project · copy, change and check on dev, then apply with a backup

The left half runs on dev as often as you like; production changes only at *Apply*, and *Undo* puts its old files back.

1.3 What the project folder holds

```
~/opennms-fundamentals/opennms-ui-studio
├── kit/                    the topology kit
|   ├── dev.sh              the development OpenNMS: up, status, seed, copy-
prod, ...
|   ├── topo.sh             the topology files: pull, deploy, view, icons,
css, ...
|   ├── kit.env             the settings: addresses, container names,
passwords of dev
|   ├── icon-types.cfg      node type = icon id (Chapter 8)
|   ├── seed/, tools/, examples/
|   └── opennms/            made by topo.sh pull: production's topology
files, to change and deploy
├── workspace/              made by the studio: your CSS, icon mappings, menu
words (a git repository)
├── backups/                made by the studio: every file an apply replaced
├── server/, web/           the studio's code
├── docs/                   how OpenNMS builds the topology page and the menu
bars
└── README.md, GUIDE.md     the studio's own manual and experiments
```

1.4 What you change in OpenNMS

All of it lives in a handful of files inside the container (Appendix A lists every one):

You want	The file in OpenNMS (below `/usr/share/opennms`)	Chapter
nodes	`etc/imports/<requisition>.xml` and the database	6
a view of your own	`etc/graphml/<name>.xml` and two `.cfg` files	7
an icon drawing	`jetty-webapps/opennms/svg/<file>.svg`	8
which node gets which icon	`etc/org.opennms.features.topology.app.icons.<name>.cfg`, or the `iconKey` of a node in its view	8
colours, frames, labels, links	your CSS, inside `jetty-webapps/opennms/svg/zz-topology-styles.svg`	9

You want	The file in OpenNMS (below `/usr/share/opennms`)	Chapter
the menu bar's words	`jetty-webapps/opennms/WEB-INF/templates/navbar.ftl` and the menu bundle `ui/assets/index-*.js`	12

None of the topology files needs a restart of OpenNMS: each one is read again when it changes, when you reload a view, or when you open a new browser tab. The menu bar has one exception, the menu names of `dispatcher-servlet.xml` (Chapter 12).

Chapter 2: OpenNMS topology, the concepts

The topology map (`/opennms/topology`) draws the nodes and links of one *view* at a time. What you see is decided by four things: the view and its data, the status of each node, the icon id of each node, and the CSS of the page.

Goal: the words and the mechanics you need for every later chapter. **You need:** nothing; one look-only exercise at the end.

2.1 Nodes

A **node** is one device OpenNMS knows: a row in its database with a **node id**, a label, interfaces, services and alarms. Nodes come from **requisitions**: lists of the nodes OpenNMS should have. A requisition has a name, its **foreign source** (for example `DAS-Demo`), and each node in it has a **foreign id** (for example `nimbus-2`). When a requisition is **imported** ("synchronize"), the provisioning daemon *provisiond* makes the database match the list and scans each node. Nodes can also come from discovery, without a requisition.

A node has two names, and the difference matters later:

Name	Example	Same on dev and production?
node id	`14`	no: each OpenNMS numbers its own nodes
foreign source and foreign id	`DAS-Demo:nimbus-2`	yes, wherever that requisition is imported

A node can also carry **categories** (labels such as `Nimbus`) and a **parent** node (`parent-foreign-id`), which the Path Outage view uses.

2.2 Views: one topology provider per View menu entry

The **View** menu of the topology page lists the views. Behind each one is a *topology provider*, a part of OpenNMS that turns some data into a graph:

View	Draws	From	Namespace
Enhanced Linkd	every node, with the links between them	the database: links found over SNMP (LLDP, CDP, OSPF, IS-IS, bridge tables) and links you define	`nodes`
Path Outage	every node under its parent	the `parent-foreign-id` of each node	`pathoutage`
a GraphML view, such as *Maven-abc* or *DAS demo*	whatever its file lists	a file in `etc/graphml/`	the one the file names
Application, Business Services, VMware	those objects	their own definitions	their own

Your production views *Maven-abc* and *MavenNebulaDAS* are GraphML views. Chapter 3 is about them.

2.3 Vertices, edges, namespace and id

On the map a node is a **vertex** and a link is an **edge**. A vertex is named by its view's **namespace** and its own **id**: `das-demo:nimbus-2` is the vertex `nimbus-2` of the view whose namespace is `das-demo`. In a GraphML view the vertex id is the `id` of the `<node>` in the file; in Enhanced Linkd it is the node id. Select a node and open the **Info** panel, *Technical Details*, to see a vertex's namespace, id and icon key.

2.4 Focus and semantic zoom

A view opens on its **focus** (some or all of its vertices) at a **semantic zoom level** (SZL): you see the focus and every vertex up to that many links away. The toolbar buttons *Increase* and *Decrease Semantic Zoom Level* change it, and the search box adds vertices to the focus. If a view shows a single node, press *Increase Semantic Zoom Level*.

2.5 Status: the coloured frame behind each icon

Behind every icon is a square, `rect.status`. Its class is the node's status, which is the severity of its worst unacknowledged alarm. The stock stylesheet colours it:

Status class	Colour	Meaning
`normal`	#336600 (the green you see everywhere)	no alarm
`warning`	#FFCC00	a warning alarm
`minor`	#FF9900	a minor alarm
`major`	#FF3300	a major alarm, such as *node down*
`critical`	#CC0000	a critical alarm

The green background behind your icons is this frame. Chapter 9 changes it with CSS.

2.6 Icons

The drawings live in svg files in `jetty-webapps/opennms/svg/`. An icon is three groups in such a file, found by their ids: `X` (normal), `X_rollover` (under the mouse) and `X_active` (selected). Each vertex gets an **icon id** `X`, chosen in this order: a line for that one vertex in an icon mapping file, else its **icon key** (in a GraphML view the `iconKey` of the node *is* the icon id), else the icon of `default` (`generic`). Chapter 8 goes through it step by step.

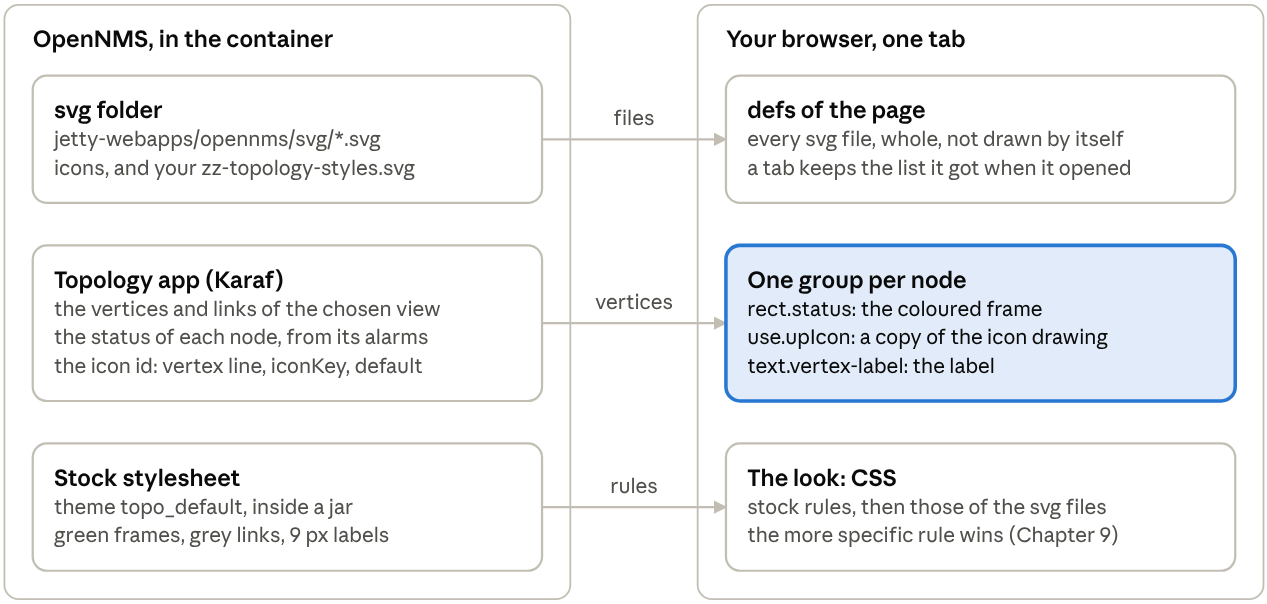
2.7 How the page is drawn

1. The browser asks for `/opennms/topology`. OpenNMS's topology app (in its Karaf container) lists every `*.svg` file of the svg folder and sends the list to the browser, once per browser tab.
2. The browser downloads each svg file and puts it, whole, into the `<defs>` of the page: definitions that are not shown by themselves.
3. The server sends the vertices of the view: position, label, status and icon id of each, and the edges.
4. The browser draws each vertex as a group of elements, and each edge as a line:

```
<g class="vertex" transform="translate(x, y)">          one node
  <rect class="status normal">                          the coloured frame
behind the icon
  <g class="icon-container">
    <use class="upIcon" href="#nimbus">                 draws the group with
id nimbus: the normal look
    <use class="overIcon" href="#nimbus_rollover">      shown while the mouse
is over the node
    <use class="activeIcon" href="#nimbus_active">      shown while the node
is selected
  <text class="vertex-label">nimbus-2</text>
<path class="edge normal">                              one link
```

5. CSS gives it its look: the stock stylesheet of OpenNMS, plus every `<style>` inside every svg file of the folder. That second source is the door for your own CSS: the kit and the studio put it into an svg file called `zz-topology-styles.svg` (Chapter 9 does it by hand).

OpenNMS sends files, vertices and rules; the browser draws the map



how the topology page is drawn · what OpenNMS sends, what the browser builds

The same steps in one picture. Three things travel from OpenNMS to your browser, and the browser does the drawing: that is why a new svg file needs a new tab, and why CSS needs no restart.

2.8 What changes when

You change	OpenNMS	You see it
add an svg file	nothing to do	in a **new browser tab**: an open tab keeps the list of svg files it got, even on reload
change an svg file	nothing to do	after a reload of the page
an icon mapping file	reads it within a second	after a reload of the page
a GraphML view file	reads it only when the view is loaded again (Chapter 3.4)	after that, and a reload
a requisition	only after an import	after the import and a reload
the stock stylesheet	it is inside a jar file	never by hand: override it with your CSS

Exercise 2.1: look at a vertex (no changes)

1. Open `http://192.168.0.47:8980/opennms/topology` and log in.
2. Open the **View** menu: note which views exist.
3. Choose *Enhanced Linkd*. Click a node, open **Info** > *Technical Details*: write down its namespace, id and icon key.
4. Press F12, choose *Elements*, press Ctrl+F and search for `class="vertex"`. Unfold one: you see the parts of 2.7.

Expect: namespace `nodes`, the id is a number, the icon key starts with `linkd.system`.

Checkpoint

- [ ] you can say what a requisition, a foreign source and a foreign id are
- [ ] you can name the views of your dev and say which ones are GraphML views
- [ ] you know why the frame behind an icon is green
- [ ] you know why a new svg file needs a new browser tab

Chapter 3: GraphML, the concepts

A GraphML view is an XML file that lists nodes and links. OpenNMS shows it as an entry of the View menu, links its nodes to OpenNMS nodes for their status, and uses each node's `iconKey` as its icon id.

Goal: read and write a GraphML view, and know when OpenNMS reads it. **You need:** nothing; you will post the file of 3.2 in Chapter 7.

3.1 GraphML in one minute

GraphML is a standard XML format for graphs. A file declares each property once with a `<key>` element, then sets it with `<data key="...">` on the file, on a `<graph>`, on a `<node>` or on an `<edge>`. A `<graph>` holds `<node>` elements (with an `id`) and `<edge>` elements (with a `source` and a `target` node id). OpenNMS reads a fixed set of properties (3.3) and ignores the rest.

3.2 A small view, line by line

This is the view you will build in Chapter 7: a site with three DAS nodes under it.

```
<?xml version="1.0" encoding="UTF-8"?>
<graphml xmlns="http://graphml.graphdrawing.org/xmlns">
  <!-- Every property used below is declared once. -->
  <key id="label" for="graphml" attr.name="label" attr.type="string"/>
  <key id="label" for="graph" attr.name="label" attr.type="string"/>
  <key id="label" for="node" attr.name="label" attr.type="string"/>
  <key id="namespace" for="graph" attr.name="namespace" attr.type="string"/>
  <key id="focus-strategy" for="graph" attr.name="focus-strategy"
attr.type="string"/>
  <key id="preferred-layout" for="graph" attr.name="preferred-layout"
attr.type="string"/>
  <key id="vertex-status-provider" for="graph" attr.name="vertex-status-
provider" attr.type="string"/>
  <key id="iconKey" for="node" attr.name="iconKey" attr.type="string"/>
  <key id="foreignSource" for="node" attr.name="foreignSource"
attr.type="string"/>
  <key id="foreignID" for="node" attr.name="foreignID" attr.type="string"/>
  <key id="level" for="node" attr.name="level" attr.type="int"/>
  <key id="tooltipText" for="edge" attr.name="tooltipText"
attr.type="string"/>
```

```
  <data key="label">Lab view (by hand)</data>              <!-- the entry in
the View menu -->
  <graph id="lab-view" edgedefault="undirected">
    <data key="namespace">lab-view</data>                <!-- required; no
other view may use it -->
    <data key="label">Lab view</data>
    <data key="focus-strategy">ALL</data>                <!-- open with
every node in focus -->
    <data key="preferred-layout">Hierarchy Layout</data>  <!-- rows by
"level", top down -->
    <data key="vertex-status-provider">default</data>    <!-- colour each
node by its OpenNMS node's alarms -->

    <!-- a drawing only: no OpenNMS node behind it -->
    <node id="site"><data key="label">Lab site</data><data
key="iconKey">cloud</data><data key="level">0</data></node>
    <!-- three nodes, each linked to an OpenNMS node of requisition LAB
(Chapter 6) -->
    <node id="lab-orion"><data key="label">lab-orion</data><data
key="iconKey">orion</data><data key="foreignSource">LAB</data><data
key="foreignID">lab-orion</data><data key="level">1</data></node>
    <node id="lab-helix"><data key="label">lab-helix</data><data
key="iconKey">helix</data><data key="foreignSource">LAB</data><data
key="foreignID">lab-helix</data><data key="level">2</data></node>
    <node id="lab-nimbus"><data key="label">lab-nimbus</data><data
key="iconKey">nimbus</data><data key="foreignSource">LAB</data><data
key="foreignID">lab-nimbus</data><data key="level">3</data></node>

    <edge id="e1" source="site" target="lab-orion"/>
    <edge id="e2" source="lab-orion" target="lab-helix"><data
key="tooltipText">Q2 - QSFP+ 40G</data></edge>
    <edge id="e3" source="lab-helix" target="lab-nimbus"><data
key="tooltipText">S1 - SFP28 25G</data></edge>
  </graph>
</graphml>
```

3.3 The properties OpenNMS reads

On	Property	What it does
the file	`label`	the entry in the View menu (without it: the view's name)

On	Property	What it does
a graph	`namespace`	names the graph; required, and unique among all views of that OpenNMS
a graph	`label`, `description`	the layer's name, a text for the Info panel
a graph	`focus-strategy`	which nodes the view opens on: `FIRST` (the default), `ALL`, `EMPTY`, or `SPECIFIC` with `focus-ids`
a graph	`semantic-zoom-level`	how many links away from the focus to show (default 1)
a graph	`preferred-layout`	a name from the page's Layout menu, such as `D3 Layout` (the default) or `Hierarchy Layout`
a graph	`vertex-status-provider`	`default`: colour each node by its OpenNMS node's alarms; without it, no status colours at all
a node	`label`, `tooltipText`	the name under the icon, the text on hover
a node	`iconKey`	the icon id itself (3.6)
a node	`foreignSource` and `foreignID`, or `nodeID`	the OpenNMS node it stands for (3.5)
a node	`level`	its row in the Hierarchy Layout
an edge	`tooltipText`	the text on hover

A file may hold several `<graph>` elements: they become *layers* of one view, each with its own namespace.

3.4 The three files of a view, and when OpenNMS reads them

You post a view to OpenNMS's REST API: `POST /opennms/rest/graphml/<name>`. OpenNMS then writes three files:

File	Says
`etc/graphml/<name>.xml`	the GraphML itself
`etc/org.opennms.features.topology.plugins.topo.graphml-<name>.cfg`	`topologyLocation` (where the xml is) and `label`: this file puts the view into the View menu
`etc/org.opennms.netmgt.graph.provider.graphml-<name>.cfg`	the same for the Graph API, `/opennms/api/v2/graphs`

OpenNMS loads the view when its first `.cfg` file appears, and reads the xml **only then**. A later change of the xml does nothing (its log says "Updating is not supported") until the view is unloaded and loaded again. Two ways to do that:

- **REST:** `DELETE /opennms/rest/graphml/<name>`, wait 5 seconds, `POST` the changed file again. There is no PUT, and a POST of a name that exists fails.
- **Files:** change the xml in the container, then move the two `.cfg` files away for 5 seconds and back. Karaf sees the view go and come back. The kit's `topo.sh view reload` does exactly this.

3.5 How a GraphML node reaches its OpenNMS node

In the node	OpenNMS uses	Between dev and production
`nodeID`	the node with that database id	not portable: on dev the same number is another node, or none
`foreignSource` and `foreignID`	the node of that requisition, looked up **when the view is loaded**	portable
neither	nothing: the node is a drawing, such as a site or a room	

`nodeID` wins when both are there. Because the lookup happens when the view is loaded, a view loaded before its nodes existed shows no status for them until it is loaded again. With `vertex-status-provider` `default`, a linked node's frame takes the colour of its worst unacknowledged alarm; a drawing stays `normal`.

3.6 The iconKey is the icon id

The `iconKey` of a GraphML node is used as the icon id as it stands: `nimbus` makes the browser draw `<use href="#nimbus">`, whatever svg file has an element with that id. It is never shortened and never looked up as a type (Chapter 8.6 explains why a mapping line for it is unreliable). A node without `iconKey` gets the `default` icon, `generic`. A node with an iconKey that no svg file defines shows no icon at all, only its frame and label.

3.7 Who writes GraphML in your setup

View	Written by	Can change at any time?
Maven-abc, *MavenNebulaDAS* (production)	your own software, one view per requisition	yes: it may write them again
DAS demo (dev only)	`kit/dev.sh seed`	when you run it
`types-<requisition>`	`kit/topo.sh view from-requisition` (Chapter 10)	when you run it
Lab view	you, in Chapter 7	when you post it

Checkpoint

- [ ] you can point at the namespace, the iconKey and the node link in the file of 3.2
- [ ] you know why changing a view's xml does nothing until the view is loaded again
- [ ] you know why `foreignSource` and `foreignID` are better than `nodeID`

Part II: By hand

Chapter 4: Your lab, the development OpenNMS

Your lab is a second OpenNMS on the development machine, built from production's image: you may change and break it, and `kit/dev.sh` rebuilds it.

Goal: the project installed, the development OpenNMS running, you logged in, the demo and production's views on dev. **You need:** Chapter 0 done; about 30 minutes, most of it waiting.

From here on, `kit$` means a terminal in the kit folder: `cd ~/opennms-fundamentals/opennms-ui-studio/kit`. The kit's own messages are written for that folder (`./dev.sh`, `./topo.sh`).

4.1 Install or update the project

Copy the package from your computer to the development machine (on your computer: `scp opennms-ui-studio-1.2.0.tar.gz admin@192.168.0.47:`), then:

```
dev$ mkdir -p ~/opennms-fundamentals && cd ~/opennms-fundamentals
dev$ cp opennms-ui-studio/kit/kit.env ~/kit.env.mine 2>/dev/null   # only if
the folder exists: your settings
dev$ tar xzf ~/opennms-ui-studio-1.2.0.tar.gz                       # makes
or updates opennms-ui-studio/
dev$ grep '"version"' opennms-ui-studio/package.json                #
"version": "1.2.0"
```

Unpacking over an existing folder keeps your work (`workspace/`, `backups/`, `kit/opennms/`, `kit/.kit/`). It replaces `kit/kit.env`: if you had changed it (a password, a port), compare it with `~/kit.env.mine` and put your values back. Instead of the full package you can unpack the updates in order (1.1.0, then 1.2.0) over a 1.0.0 folder. If a script says *Permission denied*: `chmod +x opennms-ui-studio/kit/*.sh`.

4.2 Start the development OpenNMS

```
kit$ ./dev.sh up
```

What it does, in order:

1. Checks memory (4 GB or more), disk and the ports 8980 and 8101.
2. Once, asks for your sudo password: it opens port 8980 in the firewall and keeps the containers running after you log out.
3. Once, copies production's image over ssh (about 1.5 GB; production keeps running).
4. Copies production's overlay folders, so dev runs production's code and web files (its icons among them) with its own configuration and database.
5. Starts OpenNMS and PostgreSQL 14 in the pod `onms-dev`, the way production starts its container. The first start takes 5 to 10 minutes; later ones 1 to 3.
6. Adds the demo (4.4).
7. Logs in the way a browser does, to prove that a login works.

Expect it to end like this:

```
Login at http://192.168.0.47:8980/opennms: works (admin gets /opennms/...).
Development OpenNMS: http://192.168.0.47:8980/opennms/topology   (user admin,
password admin)
Production's requisitions (its nodes) and views on dev too: ./dev.sh copy-
prod
```

`./dev.sh status` shows the state at any time; `./dev.sh down` stops the lab and `./dev.sh up` starts it again, with all its data. `./dev.sh reset` deletes it completely.

4.3 Log in

On your computer open `http://192.168.0.47:8980/opennms` and log in as admin, password admin. OpenNMS may ask you to change the default password. If you do, put the new one into `kit/kit.env` as `DEV_PASSWORD=...`: the kit logs in with it.

4.4 The demo

`./dev.sh up` adds a small DAS to dev only, so that every view has something to show:

What	Where you see it
six nodes in requisition `DAS-Demo`: orion-1; helix-1, stratus-1, stratus-2 under it; nimbus-1, nimbus-2 under helix-1	Info > Nodes; `./dev.sh status` counts them
a model number (sysObjectID) per type, and links between the nodes	View > *Enhanced Linkd*
the same DAS as a GraphML view, each node with the icon of its type	View > *DAS demo (GraphML)*
each node under its parent	View > *Path Outage* (press *Increase Semantic Zoom Level* twice)
one *node down* alarm on nimbus-2	the Alarms page, and a red frame on the maps

To make the demo again (after a kit update it happens by itself on `./dev.sh up`):

```
kit$ ./dev.sh seed
```

It prints the icon each type got, for example `nimbus nodes: iconKey nimbus`. A type without an icon of its own in the svg folder gets a stock icon (`router`, `switch`, `server`, `wifiAccess`).

4.5 Production's nodes and views on dev

```
kit$ ./dev.sh copy-prod
```

It reads production's requisitions (production is only read), sends them to dev's REST API and imports them, so dev gets the same nodes; then it copies production's views and topology files with `./topo.sh pull`, and reports how each view finds its nodes. From then on dev monitors those devices itself, so the alarms you see on dev are dev's own. `./dev.sh remove-nodes <requisition>` takes a requisition and its nodes out of dev again. Chapter 6 shows what happens inside: you will do the same by hand with a small requisition of your own.

4.6 A folder for your exercise files

The exercises of Part II write small files (a requisition, a view, an icon). Keep them together:

```
dev$ mkdir -p ~/opennms-fundamentals/lab && cd ~/opennms-fundamentals/lab
```

Checkpoint

- [ ] `./dev.sh status` says OpenNMS is *ready* (or *web UI up*)
- [ ] you are logged in at `http://192.168.0.47:8980/opennms`
- [ ] View > *DAS demo (GraphML)* shows six nodes with icons, and nimbus-2 has a red frame
- [ ] after `./dev.sh copy-prod`, the View menu also lists production's views (*Maven-abc*, *MavenNebulaDAS*)

Chapter 5: Look inside a running OpenNMS

Everything the map shows comes from files inside the container and from OpenNMS's REST API. Twenty minutes with `podman exec` and `curl` make both familiar, and nothing in this chapter changes anything.

Goal: find, read and copy the files of Appendix A, and ask the REST API. **You need:** the lab running (Chapter 4); a terminal in `~/opennms-fundamentals/lab`.

5.1 Two ways into the container

```
dev$ podman ps --format '{{.Names}}  {{.Status}}'        # onms-dev-core and
onms-dev-db (plus the pod's infra container)
dev$ podman exec onms-dev-core readlink -f /opt/opennms   #
/usr/share/opennms: OpenNMS's home
dev$ podman exec -it onms-dev-core bash                    # a shell inside
the container (sh if bash is missing); exit leaves it
```

`podman exec <container> <command>` runs one command inside; `-it` gives you an interactive shell. `/opt/opennms` is a link to `/usr/share/opennms`, so both paths work in `podman exec`.

5.2 The folders that matter

Folder, below `/opt/opennms`	Holds
`etc/`	the configuration: icon mapping files, the `.cfg` files of views, `snmp-config.xml`
`etc/graphml/`	the GraphML views
`etc/imports/`, `etc/foreign-sources/`	the requisitions as last imported, and how their nodes are scanned
`jetty-webapps/opennms/svg/`	the icon drawings, and the style file `zz-topology-styles.svg`
`jetty-webapps/opennms/WEB-INF/templates/`	`navbar.ftl`, the menu bar of the classic pages
`logs/`	`karaf.log` (topology, views), `provisiond.log` (imports), `web.log`
`system/`	OpenNMS's code; the stock topology stylesheet is inside a jar here

Exercise 5.1: list the topology files

```
dev$ podman exec onms-dev-core ls /opt/opennms/jetty-webapps/opennms/svg
dev$ podman exec onms-dev-core sh -c 'ls
/opt/opennms/etc/org.opennms.features.topology.app.icons.*'
dev$ podman exec onms-dev-core ls -l /opt/opennms/etc/graphml
/opt/opennms/etc/imports
```

Expect: the stock `opennms_icons.svg` and your own files such as `helix.svg`, `nimbus.svg` and `orion.svg`; the icon mapping files `...icons.default.cfg`, `...icons.linkd.cfg` and others; the demo view `das-demo.xml` (and production's views after `copy-prod`); the requisition `DAS-Demo.xml`. (The quotes around the second command matter: the `*` must be expanded inside the container, not on your machine.)

Exercise 5.2: read a file

```
dev$ podman exec onms-dev-core cat
/opt/opennms/etc/org.opennms.features.topology.app.icons.default.cfg
dev$ podman exec onms-dev-core grep -o 'id="[^"]*"' /opt/opennms/jetty-
webapps/opennms/svg/nimbus.svg
```

Expect: lines such as `default = generic` (the icon of a node that has no other); then the ids of `nimbus.svg`, among them `nimbus`, `nimbus_rollover` and `nimbus_active`. Chapter 8 explains why their order matters.

Exercise 5.3: copy a file out, and back in

```
dev$ podman exec onms-dev-core cat
/opt/opennms/etc/org.opennms.features.topology.app.icons.default.cfg >
icons.default.cfg.orig
dev$ ls -l icons.default.cfg.orig                  # your copy: keep it, it
is the way back for Chapter 8
```

To write a file into the container, send it through the container's own shell:

```
dev$ podman exec -i onms-dev-core sh -c 'cat > /usr/share/opennms/<path>' <
<local file>
```

This works through links and into volumes alike. For the `.cfg` files of `etc/`, write to a temporary name and rename it, so that OpenNMS never reads a half-written file (Karaf only looks at names that end in `.cfg`):

```
dev$ podman exec -i onms-dev-core sh -c 'cat >
/usr/share/opennms/etc/X.cfg.new && mv /usr/share/opennms/etc/X.cfg.new
/usr/share/opennms/etc/X.cfg' < X.cfg
```

`podman cp` works too; give it the real path, `/usr/share/opennms/...`, as the kit does.

Exercise 5.4: ask the REST API

```
dev$ curl -s -u admin:admin -H 'Accept: application/json'
'http://localhost:8980/opennms/rest/nodes?limit=0' |
       python3 -m json.tool | grep -E '"(id|label|foreignSource|foreignId)"'
dev$ curl -s -u admin:admin
http://localhost:8980/opennms/rest/requisitions/DAS-Demo
dev$ curl -s -u admin:admin http://localhost:8980/opennms/rest/graphml/das-
demo | head -20
```

Expect: every node with its id, label, foreign source and foreign id; the DAS-Demo requisition as XML; the start of the demo's GraphML. The calls this book uses:

Method and path (below `/opennms/rest`)	Does
`GET /nodes?limit=0`, `GET /nodes/<fs>:<fid>`	lists the nodes; one node by foreign source and foreign id
`POST /requisitions` (XML body)	adds or replaces a requisition (not imported yet)
`PUT /requisitions/<name>/import?rescanExisting=false`	imports it: the database follows the list
`DELETE /requisitions/<name>`, `DELETE /requisitions/deployed/<name>`	removes the pending, the deployed requisition
`POST /graphml/<name>` (XML body), `GET /graphml/<name>`, `DELETE /graphml/<name>`	adds, reads, removes a GraphML view
`POST /events` (XML body)	sends an event, such as *node down*

Exercise 5.5: read the log

```
dev$ podman exec onms-dev-core tail -n 20 /opt/opennms/logs/karaf.log
dev$ podman exec onms-dev-core grep -i graphml /opt/opennms/logs/karaf.log |
tail -5
kit$ ./dev.sh logs                    # follows karaf.log; Ctrl+C stops
(./dev.sh logs provisiond.log: imports)
```

When a view does not appear or a node shows no status, `karaf.log` usually says why.

Exercise 5.6: what the browser got

On the topology page press F12. *Network*, filter `svg`, then reload: each svg file of the folder is fetched. *Elements*, search `<defs>`: the files sit there, whole. That is the list of 2.7, step 2.

Checkpoint

- [ ] you can list the svg folder, the icon mapping files, the views and the requisitions of dev
- [ ] you have `icons.default.cfg.orig` in `~/opennms-fundamentals/lab`
- [ ] you can list the nodes of dev with `curl`

Chapter 6: Nodes, a requisition by hand

A requisition is an XML list of nodes; posting it to the REST API and importing it makes OpenNMS create exactly those nodes. You will make a requisition `LAB` with four DAS nodes that Chapters 7 to 9 use.

Goal: create, change and inspect nodes through a requisition, and give one of them an alarm. **You need:** the lab running; a terminal in `~/opennms-fundamentals/lab`.

6.1 The requisition

The nodes have no IP interfaces, so OpenNMS polls nothing; each has a category for its type and a parent, as a DAS has.

```
dev$ cat > lab.xml <<'EOF'
<model-import xmlns="http://xmlns.opennms.org/xsd/config/model-import"
foreign-source="LAB">
  <node foreign-id="lab-orion" node-label="lab-orion"><category
name="Orion"/></node>
  <node foreign-id="lab-helix" node-label="lab-helix" parent-foreign-
source="LAB" parent-foreign-id="lab-orion"><category name="Helix"/></node>
  <node foreign-id="lab-nimbus" node-label="lab-nimbus" parent-foreign-
source="LAB" parent-foreign-id="lab-helix"><category name="Nimbus"/></node>
  <node foreign-id="lab-nimbus-2" node-label="lab-nimbus-2" parent-foreign-
source="LAB" parent-foreign-id="lab-helix"><category name="Nimbus"/></node>
</model-import>
EOF
```

Part	Means
`foreign-source="LAB"`	the requisition's name: the foreign source of its nodes
`foreign-id`, `node-label`	the node's id within `LAB`, and the name OpenNMS shows
`parent-foreign-id`	the node it hangs off (Path Outage draws it there)
`<category name="Nimbus"/>`	a label; Chapter 10 uses it as the node's type

Exercise 6.1: create the nodes

```
dev$ curl -sS -u admin:admin -X POST -H 'Content-Type: application/xml' --
data-binary @lab.xml \
       http://localhost:8980/opennms/rest/requisitions
dev$ curl -sS -u admin:admin -X PUT
'http://localhost:8980/opennms/rest/requisitions/LAB/import?
rescanExisting=false'
```

The first call stores the requisition (it is *pending*); the second imports it: provisiond inserts the nodes. Wait 10 seconds, then:

```
dev$ curl -s -u admin:admin -H 'Accept: application/json'
'http://localhost:8980/opennms/rest/nodes?limit=0&foreignSource=LAB' |
       python3 -m json.tool | grep -E '"(id|label|foreignId)"'
```

Expect: four nodes, each with a node id that dev chose (write them down). In the web UI: *Info* > *Nodes* lists them; on the topology page, View > *Path Outage* shows lab-nimbus and lab-nimbus-2 under lab-helix, under lab-orion; View > *Enhanced Linkd* shows them with the `generic` icon, because they have no SNMP model (Chapter 8.5).

Exercise 6.2: the files OpenNMS wrote

```
dev$ podman exec onms-dev-core cat /opt/opennms/etc/imports/LAB.xml
```

Expect: your requisition, with a `date-stamp` and a `last-import` added. That file, not your `lab.xml`, is what OpenNMS keeps.

Exercise 6.3: a node id is not a name

Take lab-nimbus-2 out of the list, import, then put it back and import again:

```
dev$ grep -v 'lab-nimbus-2' lab.xml > lab-without.xml
dev$ curl -sS -u admin:admin -X POST -H 'Content-Type: application/xml' --
data-binary @lab-without.xml http://localhost:8980/opennms/rest/requisitions
dev$ curl -sS -u admin:admin -X PUT
'http://localhost:8980/opennms/rest/requisitions/LAB/import?
rescanExisting=false'
# wait 10 seconds: lab-nimbus-2 is gone
dev$ curl -sS -u admin:admin -X POST -H 'Content-Type: application/xml' --
data-binary @lab.xml http://localhost:8980/opennms/rest/requisitions
dev$ curl -sS -u admin:admin -X PUT
'http://localhost:8980/opennms/rest/requisitions/LAB/import?
rescanExisting=false'
```

List the nodes again (exercise 6.1). Expect: lab-nimbus-2 is back with a **new node id**, while its foreign source and foreign id, `LAB:lab-nimbus-2`, are the same. That is why a view should link its nodes by foreign source and foreign id (Chapter 3.5).

Exercise 6.4: an alarm

Send a *node down* event for lab-nimbus-2, as a failing device would cause:

```
dev$ ID=$(curl -s -u admin:admin -H 'Accept: application/json'
http://localhost:8980/opennms/rest/nodes/LAB:lab-nimbus-2 |
       python3 -c 'import json, sys; print(json.load(sys.stdin)["id"])')
dev$ echo $ID
dev$ curl -sS -u admin:admin -X POST -H 'Content-Type: application/xml' \
       --data-binary "<event xmlns=\"http://xmlns.opennms.org/xsd/event\">
<uei>uei.opennms.org/nodes/nodeDown</uei><source>book</source>
<nodeid>$ID</nodeid></event>" \
       http://localhost:8980/opennms/rest/events
```

Expect: the Alarms page lists *Node is down* for lab-nimbus-2, severity Major. Keep it: Chapter 7 shows it as a coloured frame. To clear it later, select it on the Alarms page and clear it.

What you learned

Step	What OpenNMS did
`POST /requisitions`	stored the list as a pending requisition
`PUT .../import`	made the database match the list: nodes added, and removed when missing from the list
node ids	chosen by this OpenNMS at each insert; the foreign source and foreign id stay
`POST /events`	created the event; OpenNMS turned *node down* into an alarm on the node

`./dev.sh copy-prod` (Chapter 4.5) does exactly 6.1 for each of production's requisitions.

Checkpoint

- [ ] four `LAB` nodes exist on dev, and Path Outage shows them as a tree
- [ ] you saw lab-nimbus-2 come back with a new node id
- [ ] lab-nimbus-2 has a *Node is down* alarm

Chapter 7: A GraphML view by hand

You will post the view of Chapter 3.2, see your LAB nodes in it with their alarm colour, and change it the only way OpenNMS allows: by unloading the view and loading it again.

Goal: a view of your own in the View menu, and the change cycle in your fingers. **You need:** Chapter 6 done (the LAB nodes, the alarm on lab-nimbus-2); a terminal in `~/opennms-fundamentals/lab`.

7.1 Write the file

Create `lab-view.xml` with the exact content of Chapter 3.2 (`cat > lab-view.xml <<'EOF'`, paste, `EOF`; or any editor). Its nodes use the icon ids `orion`, `helix` and `nimbus`. If your svg folder lacks one of them (exercise 5.2 shows the ids of a file), put a stock id in its place: `router`, `switch` or `wifiAccess`.

Exercise 7.1: post the view

```
dev$ curl -sS -u admin:admin -X POST -H 'Content-Type: application/xml' --
data-binary @lab-view.xml \
       http://localhost:8980/opennms/rest/graphml/lab-view
```

Reload the topology page and open the View menu. Expect: *Lab view (by hand)*. Choose it: *Lab site* on top, then lab-orion, lab-helix and lab-nimbus in rows below, each with its icon and a green frame. If the entry is missing, wait 10 seconds and reload; still missing, look in `karaf.log` (exercise 5.5).

Exercise 7.2: the files OpenNMS wrote

```
dev$ podman exec onms-dev-core ls -l /opt/opennms/etc/graphml
dev$ podman exec onms-dev-core cat
/opt/opennms/etc/org.opennms.features.topology.plugins.topo.graphml-lab-
view.cfg
```

Expect: `lab-view.xml` beside the other views, and a `.cfg` with `topologyLocation` (the path of the xml) and `label`. These are the three files of Chapter 3.4.

Exercise 7.3: a change that does nothing, then one that counts

Add the fourth node. Open `lab-view.xml` in an editor (`nano lab-view.xml`) and put these two lines just before `</graph>`:

```
    <node id="lab-nimbus-2"><data key="label">lab-nimbus-2</data><data
key="iconKey">nimbus</data><data key="foreignSource">LAB</data><data
key="foreignID">lab-nimbus-2</data><data key="level">3</data></node>
    <edge id="e4" source="lab-helix" target="lab-nimbus-2"/>
```

Now write the file straight into the container, and reload the page:

```
dev$ podman exec -i onms-dev-core sh -c 'cat >
/usr/share/opennms/etc/graphml/lab-view.xml' < lab-view.xml
```

Expect: **nothing changes**. OpenNMS read the xml when the view was loaded and does not read it again. Unload the view by moving its two `.cfg` files away for 5 seconds, then bring them back:

```
dev$ E=/usr/share/opennms/etc
dev$ podman exec onms-dev-core sh -c "mv
$E/org.opennms.features.topology.plugins.topo.graphml-lab-view.cfg
$E/org.opennms.netmgt.graph.provider.graphml-lab-view.cfg /tmp/"
dev$ sleep 5
dev$ podman exec onms-dev-core sh -c "mv
/tmp/org.opennms.features.topology.plugins.topo.graphml-lab-view.cfg
/tmp/org.opennms.netmgt.graph.provider.graphml-lab-view.cfg $E/"
```

Reload the page. Expect: lab-nimbus-2 under lab-helix. The kit's `./topo.sh view reload lab-view` does exactly these three steps.

Exercise 7.4: status colours

Look at lab-nimbus-2. Expect: its frame is red-orange (`major`), because of the *node down* alarm of exercise 6.4; the other frames are green; *Lab site* is green too, because a drawing has no node and stays `normal`. Clear the alarm on the Alarms page and reload: lab-nimbus-2 turns green.

Try it: delete the line with `vertex-status-provider` from `lab-view.xml`, write the file into the container and load the view again (7.3). Expect: no frames at all, because without a status provider a view has no status.

Exercise 7.5: focus and layout

Change `focus-strategy` from `ALL` to `FIRST` and load the view again. Expect: it opens on *Lab site* and the nodes one link away. Press *Increase Semantic Zoom Level* to see more. Change `preferred-layout` to `D3 Layout` and load it again: the same nodes, placed by a force layout instead of rows. Put both back.

Exercise 7.6: the REST cycle, and a link to nowhere

The other way to load a view again is to delete it and post it anew:

```
dev$ curl -sS -u admin:admin -X DELETE
http://localhost:8980/opennms/rest/graphml/lab-view
dev$ sleep 5
dev$ curl -sS -u admin:admin -X POST -H 'Content-Type: application/xml' --
data-binary @lab-view.xml \
       http://localhost:8980/opennms/rest/graphml/lab-view
```

Try it with a broken link: change the `foreignID` of lab-nimbus-2 to `lab-ghost`, run the cycle, reload. Expect: the node is drawn, but it has no status, and `karaf.log` says "No node found for the given foreignSource (LAB) and foreignId (lab-ghost)". Put `lab-nimbus-2` back and run the cycle again.

Keep the view: Chapter 8 uses it.

What you learned

You did	OpenNMS did
`POST /graphml/<name>`	wrote the xml and two `.cfg` files; the first `.cfg` made it load the view
changed the xml in place	nothing, until the view was loaded again
moved the `.cfg` files out and back, or DELETE and POST	dropped the view, then loaded it with the new xml
linked nodes by `foreignSource` and `foreignID`	looked them up at load time; their alarms colour the frames

Checkpoint

- [ ] View > *Lab view (by hand)* shows the site and four LAB nodes
- [ ] you have loaded a view again both ways: `.cfg` files out and back, DELETE and POST
- [ ] you saw an alarm colour a frame, and no frames without a status provider

Chapter 8: Icons by hand

Every node on the map shows one icon. In this chapter you follow how OpenNMS picks it, change it in four ways, draw an icon of your own, and meet the file layout that made your nimbus nodes light blue.

Goal: say for any node where its icon comes from, and change it on purpose: for one node, for a device model, for a whole type, and with a drawing of your own. **You need:** Chapter 7 done (View > *Lab view (by hand)* with four LAB nodes); `icons.default.cfg.orig` from exercise 5.3; about 1.5 hours.

Open a terminal in `~/opennms-fundamentals/lab` and set these once; every command of this chapter uses them (a new terminal needs them again):

```
dev$ cd ~/opennms-fundamentals/lab
dev$ C=onms-dev-core                                  # the container
dev$ E=/usr/share/opennms/etc                         # its etc folder
dev$ SVG=/usr/share/opennms/jetty-webapps/opennms/svg  # its svg folder
dev$ D=$E/org.opennms.features.topology.app.icons.default.cfg
dev$ L=$E/org.opennms.features.topology.app.icons.linkd.cfg
dev$ lab_view_reload() {     # unload the view, then load lab-view.xml again
(Chapter 7.6); run it in this folder
  curl -sS -u admin:admin -X DELETE
http://localhost:8980/opennms/rest/graphml/lab-view
  sleep 5
  curl -sS -u admin:admin -X POST -H 'Content-Type: application/xml' --data-
binary @lab-view.xml http://localhost:8980/opennms/rest/graphml/lab-view
}
```

Where a command names `helix` or `nimbus` and your svg folder has no such icon (exercise 8.1 shows), use a stock one instead: `switch` or `wifiAccess`.

8.1 An icon is three drawings with ids

An icon is not a file. It is a group with an id inside one of the svg files of `jetty-webapps/opennms/svg/`, and a node uses three of them:

The node is	OpenNMS draws	That is the group
shown	`<use class="upIcon" href="#X">`	`X`, the normal look
under the mouse	`<use class="overIcon" href="#X_rollover">`, on top	`X_rollover`
selected	`<use class="activeIcon" href="#X_active">`, on top	`X_active`

A `<use>` draws everything inside the element with that id, however deep. So the stock files keep the three looks next to each other, in a wrapper that nothing draws:

```
<g id="router_icon">                <!-- the wrapper: no <use> points at it
-->
  <g id="router_active"> ... </g>   <!-- selected -->
  <g id="router_rollover"> ... </g> <!-- under the mouse -->
  <g id="router"> ... </g>          <!-- the normal look -->
</g>
```

Two more facts. OpenNMS scales every icon so that its longest side is 48 pixels, whatever size it was drawn at. And all svg files end up in the same page, so a class such as `.st1` styles every file that uses that name: give the classes of your own drawings names of their own.

Exercise 8.1: read the ids of an icon

```
dev$ podman exec $C ls $SVG
dev$ podman exec $C grep -o 'id="router[^"]*"' $SVG/opennms_icons.svg
dev$ podman exec $C grep -o 'id="helix[^"]*"' $SVG/helix.svg
```

Expect: the stock files (`opennms_icons.svg` and a few more) and yours (`helix.svg`, `nimbus.svg`, `orion.svg`), plus `zz-topology-styles.svg` if the kit or the studio has applied CSS; then `router_icon` and the three looks of `router`; then the ids of your helix

icon. If the last command prints nothing, your file names its icon differently: `podman exec $C grep -o 'id="[^"]*"' $SVG/helix.svg` lists every id. Nothing changes in this exercise.

8.2 How OpenNMS picks the icon of a node

There is no script and no list of nodes. Each time the topology app sends the map to the browser, it asks for the icon id of every vertex and takes the first answer of three steps:

1. **A line for this one vertex**, `<namespace>.<vertex id> = <icon id>`, in any icon mapping file. Example: `lab-view.lab-nimbus-2 = helix`.
2. **Its icon key.** In a GraphML view the `iconKey` *is* the icon id, used as it stands. In the other views the key is looked up in the mapping files and, when no line matches, shortened by its last `.part` and looked up again: `linkd.system.snmp.1.3.6.1.4.1.99999.1.4`, then `linkd.system.snmp.1.3.6.1.4.1.99999.1`, and so on down to `linkd`.
3. **`default`**, the line `default = generic` of `icons.default.cfg`.

The browser then draws `<use href="#<icon id>">`. If no svg file has an element with that id, the node has no icon, only its frame and label, and OpenNMS logs nothing.

The mapping files are `etc/org.opennms.features.topology.app.icons.<name>.cfg`:

File	Read for
`...icons.default.cfg`	every view: `default`, and lines for single vertices of any view
`...icons.linkd.cfg`	Enhanced Linkd: lines for device models
`...icons.pathoutage.cfg`, `.bsm.cfg`, `.application.cfg`, `.vmware.cfg`	those views

Each of them belongs to a part of OpenNMS that registered it by its name: a file with a new name, such as `...icons.mine.cfg`, is never read. OpenNMS reads a changed file within a second; reload the page to see it. They are Java properties files: `#` starts a comment only at the start of a line, a colon or a blank inside a key needs a backslash before it, and when a key appears twice the later line counts.

Exercise 8.2: follow lab-nimbus

On the topology page choose View > *Lab view (by hand)*, click lab-nimbus and open **Info** > *Technical Details*. Expect: namespace `lab-view`, id `lab-nimbus`, icon key `nimbus`.

Step 1, a line for this vertex in any mapping file:

```
dev$ podman exec $C sh -c "grep -h '^lab-view[.]'
$E/org.opennms.features.topology.app.icons.*.cfg"
```

Expect: nothing. So step 2 decides, and the iconKey `nimbus` is the icon id. Which file has that drawing:

```
dev$ podman exec $C grep -rl --include='*.svg' 'id="nimbus"' $SVG
```

Expect: `.../svg/nimbus.svg`. In the browser, press F12, *Elements*, Ctrl+F and search `href="#nimbus"`: each nimbus node of the view has `<use class="upIcon" href="#nimbus">`. Nothing changes in this exercise.

Exercise 8.3: one node, one line

Check that your copy of exercise 5.3 is still the file:

```
dev$ diff <(podman exec $C cat $D) icons.default.cfg.orig && echo same
```

If it prints lines instead of `same`, the file has changed since then (the kit and the studio write lines into it): make a new copy, `podman exec $C cat $D > icons.default.cfg.orig`. Then add a line for lab-nimbus-2 (an empty line first, in case the file does not end with one):

```
dev$ podman exec $C sh -c "echo >> $D; echo 'lab-view.lab-nimbus-2 = helix'
>> $D"
dev$ podman exec $C tail -n 3 $D
```

Reload the page. Expect: lab-nimbus-2 shows the helix icon, lab-nimbus still the nimbus icon. A line for one vertex comes before its iconKey (step 1). Add `lab-view.lab-nimbus-2 = server` the same way and reload: the later line counts.

Undo: write your copy back. It goes to a temporary name first and is then renamed, so OpenNMS never reads half a file (it only looks at names that end in `.cfg`):

```
dev$ podman exec -i $C sh -c "cat > $D.new && mv $D.new $D" <
icons.default.cfg.orig
dev$ diff <(podman exec $C cat $D) icons.default.cfg.orig && echo same
```

Reload: lab-nimbus-2 has the nimbus icon again.

Exercise 8.4: an icon of your own

Draw it: three looks of a disc with the word LAB, in the stock layout, with class names of its own.

```
dev$ cat > lab-icon.svg <<'EOF'
<svg xmlns="http://www.w3.org/2000/svg">
  <style>
    .lab-ring        { fill: #ffffff; stroke: #1f3a5f; stroke-width: 4px; }
    .lab-ring-over   { fill: #ffe08a; stroke: #1f3a5f; stroke-width: 4px; }
    .lab-ring-active { fill: #1f3a5f; stroke: #1f3a5f; stroke-width: 4px; }
    .lab-text        { fill: #1f3a5f; font: bold 16px sans-serif; text-
anchor: middle; }
    .lab-text-active { fill: #ffffff; font: bold 16px sans-serif; text-
anchor: middle; }
  </style>
  <g id="lab_icon">
    <g id="lab_active"><circle class="lab-ring-active" cx="30" cy="30"
r="26"/><text class="lab-text-active" x="30" y="36">LAB</text></g>
    <g id="lab_rollover"><circle class="lab-ring-over" cx="30" cy="30"
r="26"/><text class="lab-text" x="30" y="36">LAB</text></g>
    <g id="lab"><circle class="lab-ring" cx="30" cy="30" r="26"/><text
class="lab-text" x="30" y="36">LAB</text></g>
  </g>
</svg>
EOF
dev$ grep -o 'id="[^"]*"' lab-icon.svg                  # lab_icon,
lab_active, lab_rollover, lab
dev$ podman exec -i $C sh -c "cat > $SVG/lab-icon.svg" < lab-icon.svg
dev$ podman exec $C ls -l $SVG/lab-icon.svg
```

Now give lab-nimbus the iconKey `lab` and load the view again. The `sed` picks the line of `<node id="lab-nimbus">` and replaces its iconKey, whatever it was; each node is one line of `lab-view.xml`, which is what lets it pick one node:

```
dev$ sed -i '/<node id="lab-nimbus">/s#<data key="iconKey">[^<]*</data>#<data
key="iconKey">lab</data>#' lab-view.xml
dev$ grep -o '<node id="lab-nimbus">.*iconKey">[^<]*' lab-view.xml    # ends
in iconKey">lab
dev$ lab_view_reload
```

Open the topology page in a **new browser tab** (an open tab keeps the list of svg files it got) and choose View > *Lab view (by hand)*. Expect: lab-nimbus is a white disc with LAB on it; under the mouse it turns yellow; selected, it is dark blue with white letters. The green in the corners is the status frame behind the icon (Chapter 2.5); Chapter 9 restyles it.

No icon at all? Check the file is there and its ids are right (the commands above), and that you opened a new tab, not a reload of the old one. Keep the icon: Chapter 9 styles it.

8.5 Enhanced Linkd: one line per model

In Enhanced Linkd the icon key of a node is `linkd.system.snmp.` followed by the model number its SNMP agent reports, its sysObjectID: `.1.3.6.1.4.1.99999.1.4` gives `linkd.system.snmp.1.3.6.1.4.1.99999.1.4`. A node without SNMP gets `linkd.system`. Since the key is shortened until a line matches (8.2, step 2), one line covers every node of a model, and a shorter one every model of a vendor:

```
linkd.system.snmp.1.3.6.1.4.1.99999.1.4 = nimbus
linkd.system.snmp.1.3.6.1.4.1.99999 = orion
```

`99999` stands for a vendor's enterprise number; the kit's demo uses it as a placeholder. A real device's key is in its Info panel, *Technical Details*: copy it from there.

Exercise 8.5: a model, one line, and Change Icon

The LAB nodes have no SNMP, so give the two nimbus nodes a model number through the REST API, as the kit's demo does (a real device gets it from its agent when OpenNMS scans it):

```
dev$ for n in lab-nimbus lab-nimbus-2; do curl -sS -u admin:admin -X PUT -H
'Content-Type: application/x-www-form-urlencoded' --data
'sysObjectId=.1.3.6.1.4.1.99999.1.4'
http://localhost:8980/opennms/rest/nodes/LAB:$n; done
dev$ curl -s -u admin:admin -H 'Accept: application/json'
http://localhost:8980/opennms/rest/nodes/LAB:lab-nimbus |
       python3 -m json.tool | grep sysObjectId
```

Wait a minute or two (Enhanced Linkd reads the nodes again in the background), reload, choose View > *Enhanced Linkd*, click lab-nimbus, Info > *Technical Details*. Expect: icon key `linkd.system.snmp.1.3.6.1.4.1.99999.1.4`, and the `generic` icon, unless a line already matches the key. Still `linkd.system`? Wait a little longer and reload.

Now one line for the model, with a copy of the file first:

```
dev$ podman exec $C cat $L > icons.linkd.cfg.orig        # your way back
dev$ podman exec $C sh -c "echo >> $L; echo
'linkd.system.snmp.1.3.6.1.4.1.99999.1.4 = nimbus' >> $L"
```

Reload. Expect: lab-nimbus and lab-nimbus-2 show the nimbus icon in Enhanced Linkd, and so do the demo's nimbus-1 and nimbus-2, which have the same model. Every future node of that model will too: one line, the whole model.

OpenNMS writes such lines itself. Right-click lab-nimbus > *Change Icon*, pick `server`, then:

```
dev$ podman exec $C cat $L
```

Expect: a new line `nodes.<node id of lab-nimbus> = server`, a line for one vertex (step 1), and a file written anew: a date on the first line, its comments gone, the lines in another order. Right-click > *Reset Icon* takes the line out again. The icons that *Change Icon* offers are listed, one per line, in `etc/org.opennms.features.topology.app.icons.list`. For a node of a GraphML view, OpenNMS documents that *Change Icon* does not apply: change its iconKey, or add a vertex line (exercise 8.3).

Undo both changes with your copy:

```
dev$ podman exec -i $C sh -c "cat > $L.new && mv $L.new $L" <
icons.linkd.cfg.orig
dev$ diff <(podman exec $C cat $L) icons.linkd.cfg.orig && echo same
```

Reload: the nimbus nodes of Enhanced Linkd have their old icon again. (The model numbers of the LAB nodes stay; they leave with the LAB nodes at the end of Chapter 9.)

8.6 Icons by node type

OpenNMS has no setting "nodes of type X get icon Y". What it has depends on the view:

View	How the type of a node picks its icon
a GraphML view	whoever writes the view puts the icon id into each node's `iconKey`: the decision is made there
Enhanced Linkd	the model: one line per sysObjectID, or a shorter one per vendor (8.5)
any view	a vertex line, for the one node that should look different (exercise 8.3)

For GraphML views the simplest rule is: **name each icon after its type.** The icon of type nimbus has the id `nimbus` (with `nimbus_rollover` and `nimbus_active`), the writer of the view puts the type into `iconKey`, and there is nothing to map. Check what your production views put into `iconKey` (`./topo.sh view show Maven-abc`, Chapter 10): when it is the type, an icon named like the type is all it takes.

What looks like a shortcut and is not: a line for the type in a mapping file, such as `helix = das_helix`. A loaded GraphML view answers for its own iconKeys too, always with the key itself, and OpenNMS asks the view and the mapping files in no fixed order (they sit in an unordered set in its code). Such a line may win today and lose after a restart or a reload of the view. If a type and its icon have different names, make them match in one place: rename the four ids in the svg file (`das_helix_icon`, `das_helix`, `das_helix_rollover`, `das_helix_active` to `helix_icon`, `helix`, `helix_rollover`, `helix_active`), or let the writer of the view put the icon id into `iconKey`.

Exercise 8.6: one decision for a whole type

In `lab-view.xml` lab-nimbus-2 still has the iconKey `nimbus` (lab-nimbus has `lab` since exercise 8.4). Give every node of the type your icon with one command (if you used `wifiAccess` in 7.1, put that word in place of `nimbus`):

```
dev$ sed -i 's#<data key="iconKey">nimbus</data>#<data
key="iconKey">lab</data>#' lab-view.xml
dev$ grep -c '<data key="iconKey">lab</data>' lab-view.xml         # 2: both
nimbus nodes
dev$ lab_view_reload
```

Reload. Expect: lab-nimbus and lab-nimbus-2 both show the LAB icon. That is all a mapping by type is in a GraphML view: one decision, made when the file is written. Chapter 10's `./topo.sh view from-requisition LAB` makes it from the categories of a requisition (`<category name="Nimbus"/>`) and one table, `kit/icon-types.cfg`.

8.7 The trap: a file that stacks its looks

A drawing program easily exports another layout: one group `X` that holds three groups, `X_default`, `X_rollover` and `X_active`. In a viewer the file looks fine. On the map, `<use href="#X">` draws everything inside `X`: all three looks, one over the other, and the one that comes last in the file lies on top. The node shows its *selected* look all the time, and a CSS rule for `#X_default` colours a layer that nobody sees. That is your `nimbus.svg`, and why its nodes are light blue.

Exercise 8.7: build the trap, then repair it

```
dev$ cat > lab-stacked.svg <<'EOF'
<svg xmlns="http://www.w3.org/2000/svg">
  <g id="stacked">
    <g id="stacked_default"><rect x="4" y="4" width="52" height="52" rx="8"
fill="#e6e7e8" stroke="#505154" stroke-width="4"/></g>
    <g id="stacked_rollover"><rect x="4" y="4" width="52" height="52" rx="8"
fill="#5dc3ad" stroke="#505154" stroke-width="4"/></g>
    <g id="stacked_active"><rect x="4" y="4" width="52" height="52" rx="8"
fill="#beccdc" stroke="#505154" stroke-width="4"/></g>
  </g>
</svg>
EOF
dev$ podman exec -i $C sh -c "cat > $SVG/lab-stacked.svg" < lab-stacked.svg
dev$ sed -i '/<node id="lab-nimbus-2">/s#<data key="iconKey">[^<]*</data>#
<data key="iconKey">stacked</data>#' lab-view.xml
dev$ lab_view_reload
```

Open a new tab, View > *Lab view (by hand)*. Expect: lab-nimbus-2 is light blue (`#beccdc` , the selected look) although nothing is selected; under the mouse it turns green; the grey of `stacked_default` never shows. Your nimbus nodes behave exactly so. The ids tell why:

```
dev$ grep -o 'id="[^"]*"' lab-stacked.svg     # stacked first;
stacked_default, _rollover and _active inside it
```

The repair renames two ids and nothing else: the group that holds the three becomes the wrapper `stacked_icon` , and `stacked_default` becomes `stacked` , the normal look:

```
dev$ sed -e 's/id="stacked"/id="stacked_icon"/' -e
's/id="stacked_default"/id="stacked"/' lab-stacked.svg > lab-stacked-
fixed.svg
dev$ diff lab-stacked.svg lab-stacked-fixed.svg   # two lines
dev$ podman exec -i $C sh -c "cat > $SVG/lab-stacked.svg" < lab-stacked-
fixed.svg
```

Reload (a changed file needs no new tab). Expect: grey; green under the mouse; light blue when selected. If the old look stays, reload with F12 open and *Network* > *Disable cache* ticked.

Put lab-nimbus-2 back on the nimbus icon for the next exercise:

```
dev$ sed -i '/<node id="lab-nimbus-2">/s#<data key="iconKey">[^<]*</data>#
<data key="iconKey">nimbus</data>#' lab-view.xml
dev$ lab_view_reload
```

Exercise 8.8: your nimbus.svg, on dev

```
dev$ podman exec $C grep -o 'id="nimbus[^"]*"' $SVG/nimbus.svg
```

If it prints `id="nimbus"` and then `id="nimbus_default"` , `id="nimbus_rollover"` , `id="nimbus_active"` , the file has the layout of 8.7. (If `id="nimbus_icon"` comes first, it was repaired already: skip this exercise.) The same two renames, with your copy as the way back:

```
dev$ podman exec $C cat $SVG/nimbus.svg > nimbus.svg.orig
dev$ sed -e 's/id="nimbus"/id="nimbus_icon"/' -e
's/id="nimbus_default"/id="nimbus"/' nimbus.svg.orig > nimbus.svg
dev$ diff nimbus.svg.orig nimbus.svg              # two lines, 20 and 22 in
your file
dev$ podman exec -i $C sh -c "cat > $SVG/nimbus.svg" < nimbus.svg
```

Reload. Expect: lab-nimbus-2, and nimbus-1 and nimbus-2 of View > *DAS demo (GraphML)*, show the grey normal look that the file's author drew; green under the mouse; light blue when selected.

This change lives in dev's container only, and dev takes production's files again when it starts (Chapter 4.2). For good, the repaired file goes to production: the studio's *Icon files* box makes the same two renames and applies them with a backup (Chapter 11), and Chapter 13 covers production. Keep the repaired file on dev for Chapter 9; its cleanup puts your copy back. To undo it now:

```
dev$ podman exec -i $C sh -c "cat > $SVG/nimbus.svg" < nimbus.svg.orig
```

What you learned

You did	OpenNMS did
a line `<namespace>.<vertex id> = <icon id>` in `icons.default.cfg`	gave that one vertex the icon within a second, ahead of its iconKey
a new svg file in the svg folder	offered its icons to every new browser tab
changed a GraphML node's `iconKey` and loaded the view again	drew the icon with that id
a line for a model in `icons.linkd.cfg`	gave every Enhanced Linkd node of that model the icon
right-click > *Change Icon*	wrote a vertex line and saved the whole mapping file anew
an svg file with the three looks inside `X`	drew all three; the last one covered the others

Checkpoint

- [ ] for each node you changed in this chapter, you can say which of the three steps of 8.2 gave it its icon
- [ ] lab-nimbus shows the LAB icon, with its own looks under the mouse and when selected
- [ ] `icons.default.cfg` and `icons.linkd.cfg` of dev match your `.orig` copies (the two `diff` commands say `same`)
- [ ] you can tell a stacked icon file by its ids and repair it with two renames

Chapter 9: The look of the map, CSS by hand

The colours, frames, labels and links of the map are CSS. In this chapter you add rules of your own to dev's topology page the way the kit and the studio do, see which rule wins and why, and colour the parts of an icon. At its end you take the lab away again: Part II is done.

Goal: change the look of the map on dev with CSS, and read in the browser which rule won. **You need:** Chapter 8 done, with its variables `C` and `SVG` set in your terminal (the first code block of Chapter 8); about 1 hour.

9.1 Where the rules come from

Source	Where it lives	Example
the stock stylesheet	inside a jar of OpenNMS (the theme `topo_default`), loaded into the page's `<head>`; you never edit it	`.vertex .status.normal { stroke: #336600; fill: #336600; }`
the `<style>` of every svg file of the svg folder	in the page's `<defs>`, where a `<style>` styles the whole page	`.cls-3 { fill: #e6e7e8; }` in your `nimbus.svg`; your own rules
what the drawing code writes on each element	attributes, and inline `style="..."`	`style="stroke-width: 3px"` on every link

The second row is the door: an svg file that holds nothing but a `<style>` adds CSS to the page. The kit and the studio put your CSS into `zz-topology-styles.svg`. In this chapter you make a file of your own, `lab-styles.svg`.

For each property of each element, the browser decides in this order:

1. A declaration with `!important` beats one without.
2. An inline `style="..."` beats every rule without `!important`.
3. Otherwise the more specific selector wins. Count the ids (`#lab`), then the classes, attributes and pseudo-classes (`.vertex` , `[href="#lab"]` , `:has(...)`), then the element types (`svg` , `rect`), and compare from the left: one more class beats any number of types.
4. With equal specificity the later rule wins. All svg files come after the stock stylesheet; among themselves they come in the order their downloads finished, which can change from one load to the next.

So the stock `.vertex .status.normal` weighs 0, 3, 0 (no id, three classes, no type), and `svg .vertex > rect.status.normal` weighs 0, 3, 2: it beats the stock rule wherever it stands. The rule of thumb that follows: **start your rules with `svg` and name the element and its classes; for the parts of an icon, start with the id of the drawing.**

The stock rules you will meet:

Element	What you see	Stock rule
`rect.status.normal`	the green square behind each icon: its fill shows through the icon, its stroke is the frame	`.vertex .status.normal { stroke: #336600; fill: #336600; }`
`rect.status.major` , `minor` , `critical` , …	the alarm colours	`.vertex .status.major { stroke: #FF3300; fill: #FF3300; }`
`text.vertex-label`	the label under the icon	`.vertex-label { font-size: 9px; text-anchor: middle; }`
`path.edge`	a link	`.edge { stroke: #ccc; }` and `.edge.up, .edge.normal { stroke: #336600; }`

Exercise 9.1: a style file of your own

The kit or the studio may have put CSS on dev already. Set it aside for this chapter, so that only the stock rules and yours meet (the cleanup at the end puts it back):

```
dev$ podman exec $C ls $SVG | grep zz-topology-styles || echo 'none: skip the
next two lines'
dev$ podman exec $C cat $SVG/zz-topology-styles.svg > zz-topology-
styles.svg.orig
dev$ podman exec $C rm $SVG/zz-topology-styles.svg
```

Your rules go into a plain CSS file, `lab.css`. A small function wraps it into an svg file and writes that into dev's svg folder, exactly as the kit does with its CSS (`<![CDATA[` lets the CSS stand as written, `>` and `&` included):

```
dev$ lab_css_apply() {   # lab.css -> lab-styles.svg -> dev's svg folder
  { echo '<svg xmlns="http://www.w3.org/2000/svg" width="0" height="0">
<style><![CDATA['
    cat lab.css
    echo ']]></style></svg>'; } > lab-styles.svg
  podman exec -i $C sh -c "cat > $SVG/lab-styles.svg" < lab-styles.svg
}
dev$ cat > lab.css <<'EOF'
/* Lab, Chapter 9: nodes without an alarm get a light background and a thin
grey frame. */
svg .vertex > rect.status.normal { fill: #F1F3F5; stroke: #8A97A8; stroke-
width: 3px; }
EOF
dev$ lab_css_apply
dev$ podman exec $C cat $SVG/lab-styles.svg
```

Open the topology page in a new tab (`lab-styles.svg` is a new file) and choose View > *Lab view (by hand)*. Press F12, and in *Network* tick *Disable cache*: while F12 is open, every reload then fetches your file anew. Keep it so for this chapter. Expect: every node on a light grey square with a grey frame instead of the green one. In View > *DAS demo (GraphML)*, nimbus-2 keeps its alarm colour, because the rule names `.normal`.

Now see why it won. In *Elements*, press Ctrl+F, search `status normal` and click the `<rect class="status normal">`. Expect, in the *Styles* pane: your rule on top, from a `<style>`; below it the theme's `.vertex .status.normal`, its `fill` and `stroke` struck through.

Exercise 9.2: a rule that loses, and one that wins

Add a pink rule written the way the stock rule is written:

```
dev$ echo '.vertex .status.normal { fill: hotpink; }' >> lab.css
dev$ lab_css_apply
```

Reload. Expect: no pink. The pink rule comes later, but your first rule weighs more (0, 3, 2 against 0, 3, 0), and weight comes before order. Now give the pink rule the same weight:

```
dev$ sed -i '/hotpink/d' lab.css
dev$ echo 'svg .vertex > rect.status.normal { fill: hotpink; }' >> lab.css
dev$ lab_css_apply
```

Reload. Expect: pink behind every node without an alarm: equal weight, so the later rule wins. Take it out again:

```
dev$ sed -i '/hotpink/d' lab.css && lab_css_apply
```

Exercise 9.3: one icon, a background of its own

```
dev$ cat >> lab.css <<'EOF'
/* the nodes drawn with the icon "lab": amber */
svg .vertex:has(use.upIcon[*|href="#lab"]) > rect.status.normal { fill:
#FFF1D6; stroke: #E08A00; }
EOF
dev$ lab_css_apply
```

Reload. Expect: lab-nimbus on amber, the other nodes light grey. Read the selector from the right: a `rect.status.normal` that is a child of a `.vertex` which has a `use.upIcon` pointing at `#lab`. `[*|href="#lab"]` matches the attribute `href` in any namespace, because OpenNMS writes `xlink:href`. It weighs 0, 5, 3 and beats the rule for all nodes.

Try it: alarm frames. Add `svg .vertex > rect.status.major, svg .vertex > rect.status.critical { stroke: #C62828; fill: #C62828; }` the same way (`cat >> lab.css`, then `lab_css_apply`) and look at nimbus-2 of the DAS demo.

Exercise 9.4: labels and links

```
dev$ cat >> lab.css <<'EOF'
svg .vertex > text.vertex-label { font-size: 12px; font-weight: bold; fill:
#1F3A5F; }
/* OpenNMS writes stroke-width: 3px into each link (style=""), so the width
needs !important */
svg path.edge, svg path.edge.normal { stroke: #1F3A5F; stroke-width: 4px
!important; }
EOF
dev$ lab_css_apply
```

Reload. Expect: bigger, bold, dark blue labels; dark blue links, 4 pixels wide. Then edit `lab.css` (`nano lab.css`), delete `!important`, apply and reload: the links are 3 pixels again, because the inline style beats every rule without `!important`. Put it back. The rule names `svg path.edge.normal` as well because `svg path.edge` alone (0, 1, 2) loses to the stock `.edge.normal` (0, 2, 0): classes count before types.

9.2 The parts of an icon

A drawing has its own colours, in its file: in attributes, or in classes of its own `<style>`. A rule for its parts starts with the id of the drawing that is shown, `#X .class`. It reaches every copy that a `<use>` makes of that drawing, and with its weight of 1, 1, 0 it beats every class rule of every svg file.

That matters for files from a drawing program. Illustrator names the classes of every file it exports `cls-1`, `cls-2`, and so on, and a file's `<style>` styles the whole page: when two icon files both define `.cls-1`, the file that arrived last colours both. `#nimbus .cls-1` is the `cls-1` of the nimbus drawing only. (The kit's `./topo.sh css` lists the classes that several files define differently; Chapter 10.)

Exercise 9.5: the parts of your LAB icon

```
dev$ echo '#lab .lab-ring { fill: #FFE9A8; }' >> lab.css
dev$ lab_css_apply
```

Reload. Expect: the disc of lab-nimbus turns light amber. Under the mouse it is still yellow, and selected still dark blue: those are the drawings `#lab_rollover` and `#lab_active`, with classes of their own, which a rule for `#lab` does not reach.

Exercise 9.6: the nimbus case, in CSS

Your `nimbus.svg` colours its normal look with the classes `cls-1` to `cls-3`, and the other two looks with `rollover-cls-N` and `active-cls-N`:

```
dev$ podman exec $C grep -o 'class="[^"]*"' $SVG/nimbus.svg | sort | uniq -c
```

Put the repaired file of exercise 8.8 on dev (again), and colour its main class:

```
dev$ podman exec -i $C sh -c "cat > $SVG/nimbus.svg" < nimbus.svg
dev$ echo '#nimbus .cls-3 { fill: #FFB74D; }' >> lab.css
dev$ lab_css_apply
```

Reload. Expect: lab-nimbus-2 and the demo's nimbus nodes turn orange where they were light grey; under the mouse and selected they show their other two looks. Now put the original file back, as production still has it:

```
dev$ podman exec -i $C sh -c "cat > $SVG/nimbus.svg" < nimbus.svg.orig
```

Reload. Expect: light blue again, and no orange. The rule still matches the `cls-3` shapes, but in the original file they lie under `#nimbus_active`, the last of the three looks (Chapter 8.7). That is what happened to `#nimbus_default .cls-1 { fill: red !important; }` on your OpenNMS: the right shapes, but covered. A rule for the layer on top does show:

```
dev$ echo '#nimbus_active .active-cls-3 { fill: #FFB74D; }' >> lab.css
dev$ lab_css_apply
```

Reload. Expect: orange, now when selected too, because with this file both the normal and the selected look draw `#nimbus_active`. It works, but it is a workaround; the repair of the file (8.8, and the studio in Chapter 11) is the fix. (If your `nimbus.svg` was repaired before exercise 8.8, the second half of this exercise does not apply.)

9.3 Clean up the lab

Part II ends here. Take the lab out of dev, so that from now on dev shows only production's state and the work of the kit and the studio. The files:

```
dev$ podman exec $C rm $SVG/lab-styles.svg $SVG/lab-icon.svg $SVG/lab-
stacked.svg
dev$ podman exec -i $C sh -c "cat > $SVG/nimbus.svg" < nimbus.svg.orig
# if you made it in 8.8
dev$ podman exec -i $C sh -c "cat > $SVG/zz-topology-styles.svg" < zz-
topology-styles.svg.orig   # if you set it aside in 9.1
dev$ curl -sS -u admin:admin -X DELETE
http://localhost:8980/opennms/rest/graphml/lab-view
```

The LAB nodes: an import of the requisition without nodes deletes them; then the requisition itself goes:

```
dev$ echo '<model-import xmlns="http://xmlns.opennms.org/xsd/config/model-
import" foreign-source="LAB"/>' > lab-empty.xml
dev$ curl -sS -u admin:admin -X POST -H 'Content-Type: application/xml' --
data-binary @lab-empty.xml http://localhost:8980/opennms/rest/requisitions
dev$ curl -sS -u admin:admin -X PUT
'http://localhost:8980/opennms/rest/requisitions/LAB/import?
rescanExisting=false'
dev$ sleep 10
dev$ curl -sS -u admin:admin -X DELETE
http://localhost:8980/opennms/rest/requisitions/LAB
dev$ curl -sS -u admin:admin -X DELETE
http://localhost:8980/opennms/rest/requisitions/deployed/LAB
dev$ curl -s -u admin:admin -H 'Accept: application/json'
'http://localhost:8980/opennms/rest/nodes?limit=0&foreignSource=LAB' |
python3 -m json.tool | grep '"count"'
```

Expect: `"count": 0`. In a new tab, the View menu has no *Lab view (by hand)*, and `podman exec $C ls $SVG` lists no lab files. The files in `~/opennms-fundamentals/lab` stay: they are the record of what you did. The kit takes a requisition and its nodes out with one command, `./dev.sh remove-nodes LAB` (Chapter 10).

What you learned

You did	What happened
put an svg file that holds only a `<style>` into the svg folder	its rules joined the CSS of the page, in every new tab

You did	What happened
wrote a rule the way the stock rule is written	it lost: the browser compares weight before order
started rules with `svg` and named element and classes	they beat the stock rules wherever they stood
put `!important` on the width of links	it beat the inline style that OpenNMS writes
wrote a rule for the parts of an icon, starting with its id	it reached every node that shows that drawing, unless another look lay on top

Checkpoint

- [ ] you can name the three sources of rules on the topology page, and find with F12 which rule won for an element
- [ ] you can say why `.vertex .status.normal { fill: hotpink; }` changed nothing
- [ ] you can write a rule for the nodes of one icon, and one for the parts of an icon
- [ ] dev is clean: no Lab view, no LAB nodes, no lab files in the svg folder; your `nimbus.svg` and, if you had one, `zz-topology-styles.svg` are back

Part III: The tools

Chapter 10: The topology kit

The kit is the command-line half of the project: shell and Python scripts in `kit/` that do the steps of Part II for you, on dev and on production, and check what they did. Every command in this chapter is one or more exercises of Part II in one line.

Goal: run the kit's commands, and say for each one which steps of Part II it does. **You need:** Part II done; a terminal in the kit folder (`kit$` , Chapter 4); about 1 hour.

10.1 From Part II to one command

By hand	Chapter	The kit
copy the topology files out of production's container, and into dev's	5	`./topo.sh pull` : into `opennms/` , then onto dev

By hand	Chapter	The kit
post a requisition and import it	6	`./dev.sh copy-prod [requisition...]`, for production's requisitions
an import without nodes, then DELETE	9.3	`./dev.sh remove-nodes <requisition>`
post a GraphML view	7.1	`./topo.sh view add <file> [name]`
move a view's two `.cfg` files out and back	7.3	`./topo.sh view reload [name...]`
list the views; follow the icon of a node	8.2	`./topo.sh view list`, `./topo.sh view show <name>`
a line for one vertex in `icons.default.cfg`	8.3	`./topo.sh view icon <name> <node id> <icon id>` (`--reset` takes it out)
read the ids of the svg files; spot the stacked layout	8.1, 8.7	`./topo.sh icons`
a view whose icons follow the node type	8.6	`./topo.sh view from-requisition <requisition>` with `icon-types.cfg`
wrap CSS into an svg file and copy it in	9.1	`./topo.sh deploy dev` (or `./topo.sh watch`)
find classes that clash; rules for the parts of an icon	9.2	`./topo.sh css`, `./topo.sh css <icon id>`
copy changed files into production, with a backup	13	`./topo.sh deploy prod`, `./topo.sh undo`

`opennms/` is the kit's working copy of the topology files. `./topo.sh pull` fills it with production's files; you change files there; `./topo.sh deploy dev` or `deploy prod` copies what changed into that container.

10.2 The kit's folder

```
kit/
├── dev.sh, topo.sh       the two commands; either one without a word lists
what it can do
├── kit.env               settings: addresses, container names, dev's login,
where your CSS is
├── icon-types.cfg        node type = icon id (Chapter 8.6)
├── opennms/              production's topology files as of the last pull:
change them here
├── reference/            the stock stylesheet of the topology page, for
reading
├── .kit/                 the kit's memory: production's files at the last
pull, copies of the overlay folders
├── backups/              what each production deploy replaced
├── seed/, examples/, tools/
└── README.md, FILES.md, GUIDE-*.md
```

Your CSS lives outside the kit: `STYLE_CSS` in `kit.env` points at the studio's `workspace/topology/topology.css`, so the kit and the studio share one CSS file.

What the kit may change: on dev, anything it is asked to; on production, only `./topo.sh deploy prod` (it asks first and keeps a backup), `./topo.sh undo` and `./topo.sh view remove <name> prod`. `pull`, `css prod`, `icons prod`, `persist` and `./dev.sh copy-prod` only read production.

Exercise 10.1: pull, and look at what came

`./topo.sh pull` replaces what is in `opennms/`, so look first whether you changed anything there:

```
kit$ ./topo.sh diff
kit$ ./topo.sh pull
kit$ find opennms -type f | sort
```

Expect: `diff` reports no changes (or lists yours: then keep them before you pull). `pull` copies production's topology files and puts them onto dev as well. The list shows the folders of Chapter 5.2 in miniature: the icon mapping files and the Change Icon list in `opennms/etc/`, the views in `opennms/etc/graphml/` with their `.cfg` files, the svg files in `opennms/jetty-webapps/opennms/svg/`. The demo view is not among them: it belongs to dev. (`./dev.sh copy-prod` in Chapter 4.5 ran a pull already.)

Exercise 10.2: views, and the icon of each node

```
kit$ ./topo.sh view list
kit$ ./topo.sh view show das-demo
kit$ ./topo.sh view show Maven-abc
```

Expect, for the demo, a table like this one (your icons depend on your svg files):

```
View das-demo in the development OpenNMS, with the icon mappings of
opennms/etc:
View menu label: DAS demo (GraphML)
Layer "DAS demo": namespace das-demo; 6 nodes, 5 links; layout Hierarchy
Layout, focus ALL, zoom level 1, status default
  node id          label                       iconKey              icon
drawn
  orion-1          orion-1                     orion                orion
(its iconKey)
  helix-1          helix-1                     helix                helix
(its iconKey)
  nimbus-2         nimbus-2                    nimbus               nimbus
(its iconKey)
```

The last column says which step of 8.2 decided: `(its iconKey)` is step 2; a vertex line names its mapping file (step 1); `default` is step 3. For *Maven-abc* the table shows what production's software writes into `iconKey`, and flags a node whose icon no svg file defines: such a node shows no icon.

Exercise 10.3: the icon files, and the stacked trap

```
kit$ ./topo.sh icons
```

Expect: each svg file with its icon ids; for `nimbus.svg`, unless it was repaired, a warning that the group `nimbus` holds `nimbus_default`, `nimbus_rollover` and `nimbus_active`, so a node shows `nimbus_active`, the last one (Chapter 8.7). Then every view with the icon each node gets and why. `./topo.sh icons prod` does the same for production.

Exercise 10.4: a view whose icons follow the node type

```
kit$ cat icon-types.cfg
kit$ ./topo.sh view from-requisition DAS-Demo
```

Expect: the table it decided by, for example:

```
Requisition DAS-Demo of the development OpenNMS, with icon-types.cfg:
  requisition DAS-Demo: 6 nodes, 5 links to a parent
  node label                  foreign id          type (from)
icon (iconKey)
  orion-1                     orion-1             orion (category Orion)
orion
  helix-1                     helix-1             helix (category Helix)
helix
  nimbus-2                    nimbus-2            nimbus (category Nimbus)
nimbus
```

For each node it found the type (its first category that `icon-types.cfg` names, else its asset category or model number, else the first word of its label), wrote the icon of that type as the node's iconKey, linked the node by foreign source and foreign id, and added the view to `opennms/` and to dev. A type without an icon of its own gets a line starting with `!` that says what it got instead. Open a new tab: View > *DAS-Demo: icons by node type*.

Now change the icon of a whole type in one place: edit `icon-types.cfg` so that it says `nimbus = helix`, run `./topo.sh view from-requisition DAS-Demo` again and reload. Expect: every Nimbus node of the view shows the helix icon. Put the line back to `nimbus = nimbus` and run it once more. After `./dev.sh copy-prod` you can do the same with one of production's requisitions (a wrong name lists the requisitions dev has).

Exercise 10.5: one node, one line

The view of 10.4 is in `opennms/`, so the kit can write a vertex line for one of its nodes (it does that for the views in `opennms/`, not for the demo view, which only dev has):

```
kit$ ./topo.sh view icon types-DAS-Demo nimbus-1 helix
kit$ tail -n 2
opennms/etc/org.opennms.features.topology.app.icons.default.cfg
kit$ podman exec onms-dev-core tail -n 2
/opt/opennms/etc/org.opennms.features.topology.app.icons.default.cfg
```

Expect: a comment and the line `types-DAS-Demo.nimbus-1 = helix`, in both files: the kit wrote it in `opennms/` and deployed the file to dev. Reload: nimbus-1 shows the helix icon, nimbus-2 still its type's icon. That was exercise 8.3. Take the line out, then the view, which would otherwise go to production with the next `./topo.sh deploy prod`:

```
kit$ ./topo.sh view icon types-DAS-Demo nimbus-1 --reset
kit$ ./topo.sh view remove types-DAS-Demo
```

Exercise 10.6: the CSS checks

```
kit$ ./topo.sh css
kit$ ./topo.sh css nimbus
```

Expect: the svg files, the style rules of theirs that clash (the same class defined differently by two files, Chapter 9.2), whether dev has your CSS and what its web server answers for it; then CSS for the parts of the nimbus icon, made from its file, ready to copy into your CSS. The CSS itself you will edit in the studio (Chapter 11), which writes the same file `./topo.sh deploy dev` sends.

What you learned

The kit	does what you did in
`./topo.sh pull`, `./dev.sh copy-prod`, `./dev.sh remove-nodes`	Chapters 5, 6 and 9.3
`./topo.sh view add`, `view reload`, `view show`	Chapters 7 and 8.2
`./topo.sh view from-requisition`, `view icon`, `icons`	Chapters 8.6, 8.3 and 8.7
`./topo.sh css`, `deploy dev`	Chapter 9

More: `README.md` of the kit (all commands), `FILES.md` (every file, with an example to run: `./topo.sh example`), `GUIDE-node-icons.md` (icons), `GUIDE-icon-backgrounds.md`, `GUIDE-production-permanent.md`.

Checkpoint

- [ ] `opennms/` holds production's topology files, and `./topo.sh diff` knows what you changed there
- [ ] you can read the output of `./topo.sh view show` and say which step gave each node its icon
- [ ] you made and removed a view by node type, and know where to change the icon of a type

- [ ] you know which kit commands may change production, and which only read it

Chapter 11: OpenNMS UI Studio

The studio is a web page for the work of Chapters 8 and 9: your CSS with a live preview of the map, the icon of each node, a check of the icon files, and Apply and Undo for dev and production, each with a backup. Every version of your files is kept in git, and every command the studio runs is shown as you would type it.

Goal: the studio running, your old CSS in it, one change applied to dev and taken back, and your files under version control. **You need:** Chapters 9 and 10; Node.js 20 or newer and git (Chapter 0.3); about 1.5 hours.

11.1 Install and start

```
dev$ cd ~/opennms-fundamentals/opennms-ui-studio
dev$ npm install          # once, and again when an update says so: Vue, Vite
and TypeScript into node_modules/
dev$ npm run build        # after every update: compiles the server and the
page
dev$ npm test             # 21 tests, all "ok"
dev$ sudo firewall-cmd --permanent --add-port=8990/tcp && sudo firewall-cmd -
-reload   # once
dev$ npm start
```

Expect it to print the address with an access token:

```
OpenNMS UI Studio is running. Open in a browser:

  http://192.168.0.47:8990/?token=3f9c...
```

Open that address in your browser. The token is then kept in a cookie, so later `http://192.168.0.47:8990/` is enough. Ctrl+C in the terminal stops the studio; your work stays in `workspace/`. The first start creates `workspace/`, a git repository with your files, and makes its first snapshot. (To keep the studio running without a terminal: README.md, section 8.)

11.2 A tour: four pages and the command log

Page	What it is for
Start	the state of dev, production and your workspace; two dots in the left column, green when the studio reaches dev and production
Topology	your CSS with a live preview of the map; the icon of each node; the check of the icon files; Apply and Undo for the topology
Menu bar	the words of the menu bar (Chapter 12)
History	snapshots and experiments of your files, compare, and every apply with its checks

At the bottom of every page is the **command log**: each command the studio runs, `podman ...` for dev, `ssh maven-admin@192.168.0.36 podman ...` for production, `git ...` for your workspace, with its exit code and time. Open it and keep it open while you learn: it is Part II, played back.

If production's dot is red, the text under it says why; usually `ssh` asks for a password (Chapter 0.3).

11.3 Bring your CSS from the old kit

Your earlier rules are in the retired kit folder. Bring them over once, before you change anything in the editor, because an import replaces the CSS of the editor:

1. **Topology** > *Import a CSS file (for example the one of your old topology kit)*.
2. Enter `~/opennms-fundamentals/opennms-fundamentals/opennms-topology-kit-v6/styles/topology.css` and press *Import*.
3. **History** > message `My CSS from the topology kit` > *Snapshot*.

Expect: the editor shows your rules, the preview draws them, and History lists the snapshot. The file behind the editor is `workspace/topology/topology.css`, the same file the kit deploys (Chapter 10.2). If you have no old CSS, skip this section.

11.4 The preview and the inspector

The preview is built the way OpenNMS builds the topology page (Chapter 2.7): the stock stylesheet, every svg file of dev in `<defs>`, your CSS as the last of them, and the nodes of a GraphML view of dev. Under the editor, *What your rules reach in this preview* says, for each of your selectors, what it matches and whether another shape covers it. Under the preview, *Icon files* checks the svg files.

Exercise 11.1: ask the preview what you asked F12 in Chapter 9

1. Choose the view *DAS demo (GraphML)* for the preview, and *Status > no alarms (normal)*.
2. Click nimbus-2. Expect, in the inspector: why it has its icon (for example *because of its iconKey in the GraphML file*: step 2 of 8.2), and what the browser computes for its frame: fill, stroke, stroke-width, and the size and colour of its label.
3. Open *What your rules reach in this preview*. Expect: each selector of your CSS with the elements it matches; a selector that matches nothing says so, and a rule whose shapes lie under another shape says which drawing is on top.

Nothing changes in OpenNMS: the command log shows that the studio only read dev's files.

11.5 Change, apply, undo

Exercise 11.2: one rule, applied to dev, then undone

1. *Insert rules to start from… > All icons: another background instead of the green > Insert*. The preview changes as the line appears; change its colour and watch it again.
2. In the panel *Development OpenNMS*, read the plan: one file, `jetty-webapps/opennms/svg/zz-topology-styles.svg`, its md5 now and after, and its diff: your CSS between `<style><![CDATA[` and `]]></style>`. That is the file you made by hand as `lab-styles.svg` in 9.1.
3. *Apply to the development OpenNMS*. Read the checks: the md5 of the file in the container, and *OpenNMS serves svg/zz-topology-styles.svg … 200, the new file*.
4. Open dev's topology page in a new tab: your colour.
5. Check it by hand, in a second terminal (the first one runs the studio):

```
dev$ cd ~/opennms-fundamentals/opennms-ui-studio
dev$ podman exec onms-dev-core head -5 /usr/share/opennms/jetty-
webapps/opennms/svg/zz-topology-styles.svg
dev$ ls backups/ && tail -n 1 backups/journal.jsonl | python3 -m json.tool |
head -20
```

6. *Undo the last topology apply on dev*. Reload: the colour is gone, and the command log shows the old file going back. Apply again if you want the rule on dev.

What happened in OpenNMS: `podman exec onms-dev-core cat <file>` (the old file, into `backups/<date>-<time>-dev-topology/`), `podman exec -i onms-dev-core sh -c '... cat > <file>'` (the new one), `md5sum`, then a request for `svg/zz-topology-styles.svg`. Nothing restarted. Every apply also makes a snapshot, so the journal of applies names the exact version dev got.

11.6 Icons in the studio

Exercise 11.3: another icon for one node

1. Click a node of a view, for example helix-1 of *DAS demo (GraphML)*. In the inspector, *Another icon for this node* > pick an icon. The preview draws it at once, and `workspace/topology/icon-mappings.json` gets the line.
2. The dev panel now plans two files: the style file and `etc/org.opennms.features.topology.app.icons.default.cfg`, whose diff adds a block between two comments of the studio. Apply to dev and reload the page.
3. Check it by hand: `podman exec onms-dev-core tail -6 /usr/share/opennms/etc/org.opennms.features.topology.app.icons.default.cfg`. Expect: your line `das-demo.helix-1 = <icon>` between the studio's two comments: exercise 8.3, written for you.
4. Back: *Another icon for this node* > *(no line of the studio)*, and apply again.

Exercise 11.4: the nimbus file, repaired and applied

1. Open *Icon files*, under the preview. Expect: each svg file of dev as a card with its icons, *fine* or *has a flaw*. For `nimbus.svg` it says, in words, what you found in 8.7: `#nimbus` holds the three looks, and `#nimbus_active` lies on top.
2. If your CSS has rules for `#nimbus_default` (many old ones do), *What your rules reach* says that their shapes lie under a later shape: the drawing on top is `#nimbus_active`. That is exercise 9.6.

3. *Make a repaired copy in the workspace*. Expect: the preview draws your copy (the inspector says so); the nimbus nodes show the grey of the normal look. The copy is `workspace/topology/icons/nimbus.svg`: the file with two ids renamed, exactly the `sed` of exercise 8.8.
4. Rules for `#nimbus_default` now match nothing, because that id is gone. Write them for `#nimbus` instead, for example `#nimbus .cls-3 { fill: #FFB74D; }`.
5. The dev panel lists a second file, `svg/nimbus.svg`, with a diff of two lines. Apply to dev, open a new tab, and check by hand: `podman exec onms-dev-core grep -o 'id="nimbus[^"]*"' /usr/share/opennms/jetty-webapps/opennms/svg/nimbus.svg` prints `nimbus_icon` first.

Production gets the repaired file with your next production apply (Chapter 13), which is the fix the nimbus nodes have waited for.

11.7 History: snapshots and experiments

The History page is git with buttons:

You want	In the studio	The same in a terminal, in `workspace/`
keep a version	message, then *Snapshot*	`git add -A && git commit -m "..."`
try an idea without losing the current one	*New experiment*	`git switch -c experiment/<name>`
go back to another line	*Switch* (snapshot your changes first)	`git switch <branch>`
see the difference of two versions	pick two, *Compare the two picked*	`git diff <older> <newer>`
take the files of an old version back	*Take its files back*, then snapshot	`git restore --source=<snapshot> -- topology menu`
know what each OpenNMS got	*Applies*, each with the snapshot it applied	`cat ../backups/journal.jsonl`

Exercise 11.5: an experiment

1. *New experiment* `amber-helix`. The left column says `experiment/amber-helix`.

2. Click a helix node; *Insert rules to start from… > One icon: a background of its own > Insert*. The rule names the icon you clicked, as in exercise 9.3. Snapshot: `amber behind helix`.
3. *Switch* to `main`: the editor and the preview go back to the version without amber. *Switch* to `experiment/amber-helix`: amber again.
4. Pick the two snapshots, *Compare the two picked*: the diff is your one new line.
5. Switch back to `main` (or keep the experiment as your new main line: GUIDE.md, part 4).

What you learned

In the studio	By hand, in
the preview and *What your rules reach*	F12 on the real page, Chapter 9
Apply to dev: the style file, the mapping block, a repaired icon file	exercises 9.1, 8.3 and 8.8
Undo	your `.orig` copies
Icon files	the ids of 8.1 and 8.7
History	git in `workspace/`

The studio's own course, GUIDE.md, has more experiments (T1 to T8 for the topology); docs/how-the-studio-works.md lists every command it runs.

Checkpoint

- [ ] the studio runs, and both dots on its Start page are green
- [ ] your old CSS is in the editor, kept as a snapshot
- [ ] you applied a change to dev, found the file in the container, and undid it
- [ ] dev has the repaired `nimbus.svg`, applied from the studio (or *Icon files* says the file is fine)
- [ ] you switched between two snapshots of an experiment

Chapter 12: The menu bar

The second, smaller module of the project: the words of the menu bar at the top of every OpenNMS page. You renamed *Plugins* to *Tools* once, and some pages kept saying *Plugins*. This chapter shows why, and how the studio changes the word in every place at once.

Goal: know which file decides each word of the bar, and change the Plugins menu and a menu name on dev. **You need:** Chapter 11 (the studio running); a terminal on the development machine; about 45 minutes.

12.1 One bar, two programs

The bar looks the same on every page, but two programs make it, from different files, at different times:

Pages	The bar is made by	From	Read again
the JSP pages: `index.jsp`, nodes, alarms, admin, and the topology map	the server, which fills the template `navbar.ftl`	`WEB-INF/templates/navbar.ftl`, and the menus of `WEB-INF/dispatcher-servlet.xml`	the template within about 5 seconds of a change; the menus only when OpenNMS starts
the Vue pages: `/opennms/ui/#/...`, every plugin page among them	the browser, running the compiled `Menubar.vue`	`ui/assets/index-<hash>.js`, the menus from `/opennms/api/v2/menu`, the plugin entries from `/opennms/rest/plugins`	at every page load; `/api/v2/menu` reads `dispatcher-servlet.xml` at every request

All paths are below `jetty-webapps/opennms/`. So most words live in one place for both bars (the menus of `dispatcher-servlet.xml`), but *Plugins* lives in two: line 94 of `navbar.ftl` for the JSP bar, and the compiled `Menubar.vue` for the Vue bar. Change one of the two, and the bars disagree. That is what happened to *Tools*.

Word	JSP bar	Vue bar
Info, Status, Reports, Dashboards, Maps and their entries	`dispatcher-servlet.xml`	the same file, through `/api/v2/menu`
Plugins	`navbar.ftl`, line 94	`Menubar.vue`, compiled into `ui/assets/index-<hash>.js`
Help, the user menu, the icons at the right	`navbar.ftl`	Java code of OpenNMS (`MenuProvider.java`), compiled into a jar
Search	`dispatcher-servlet.xml`	written by `Menubar.vue` itself: keep the name

Exercise 12.1: find the words by hand

```
dev$ C=onms-dev-core; W=/usr/share/opennms/jetty-webapps/opennms
dev$ podman exec $C grep -n 'nav-plugin-top' $W/WEB-INF/templates/navbar.ftl
dev$ podman exec $C grep -o 'assets/index-[0-9a-zA-Z_-]*[.]js'
$W/ui/index.html
dev$ podman exec $C sh -c "cd $W/ui/assets && grep -l menubar-dropdown-
button-dark *.js"
dev$ curl -s -u admin:admin http://localhost:8980/opennms/api/v2/menu |
python3 -m json.tool | grep '"name"' | head
```

Expect: the line of `navbar.ftl` with the word of the JSP bar; the bundle that `ui/index.html` loads; the bundles that hold the compiled bar (there can be more than one, an older one that nothing loads); the menu names that the Vue bar gets. In a compiled bundle the word is a string such as `" Plugins "` right after the class `menubar-dropdown-button-dark`: that is what the studio finds and changes.

Exercise 12.2: your word for the Plugins menu

1. In the studio, **Menu bar** > *The Plugins menu*: type your word, for example `Tools`. Both bars of the live preview show it at once.
2. The dev panel plans `navbar.ftl` with a one-line diff, and each bundle that holds the compiled bar, with the text around the change (a bundle is one long line).

3. Apply to dev. The checks: the md5 sums, the web server sending the bundles with the new word, and *The JSP bar (/opennms/navBar.htm) says the new Plugins label*.
4. Reload `http://192.168.0.47:8980/opennms/index.jsp` and a plugin page. Expect: your word in both bars.
5. Run the commands of 12.1 again: the new word in each file.

Nothing restarted: FreeMarker reads a changed template again, and the browser loads the bundle again with the page.

Exercise 12.3: a menu name, and the one restart

1. In *Menu names from dispatcher-servlet.xml*, give `Maps` the name `Network Maps`, and apply to dev.
2. Open a Vue page: `Network Maps`. Open `index.jsp`: still `Maps`. The checks of the apply say the same: the Vue bar has the new name, the JSP bar gets it after the next restart.
3. *Restart the development OpenNMS* (a few minutes; the command log shows `podman restart onms-dev-core`). Reload `index.jsp`: `Network Maps`.
4. Put the name back the same way if you only wanted to see it.

Why: both bars read the menu names from `dispatcher-servlet.xml`, but the JSP bar makes its menus once, when OpenNMS starts, and the Vue bar asks `/api/v2/menu`, which reads the file at every request.

12.2 Words to leave alone

Help, the user menu and the titles of the icons at the right come from `navbar.ftl` for the JSP bar, but from Java code compiled into a jar for the Vue bar. Changing only `navbar.ftl` would make the bars disagree, the very problem *Plugins* had, so the studio leaves them alone and lists them under *Words the studio does not change*. *Search* keeps its name: the Vue bar finds the search by that name.

The files of this module are OpenNMS's own files, not yours. On production the studio also puts copies into the overlay folder, so that a restart keeps them; before you upgrade OpenNMS to a new image, those copies have to go (Chapter 13.6).

Checkpoint

- [ ] you can say why the JSP bar and the Vue bar disagreed about *Tools*
- [ ] dev shows your word for the Plugins menu on a JSP page and on a Vue page

- [ ] you know which change needs a restart of OpenNMS, and for which bar

Part IV: Production

Chapter 13: Production, safely

Everything so far happened on dev. This chapter takes the finished work to production: the same files, written the same way, plus a copy in production's overlay folders so that a restart keeps them, a backup of everything replaced, and an undo.

Goal: your CSS, icon mappings, repaired icon files, views and menu words on production, kept at every restart, with a way back. **You need:** dev looks right in a new tab; Chapters 10 to 12; a quiet moment on production; about 1 hour.

13.1 What lasts where

A file can be in four places, and each survives different things:

Where the file is	Put there by	A restart of the container	A new container	A new machine
inside the running container (`/usr/share/opennms/...`)	an apply, `./topo.sh deploy prod`, `podman cp`	kept, unless an overlay copy replaces it	lost	lost
an overlay folder on the host (`/opt/maven-nms/data/overlays/...`)	an apply, `./topo.sh deploy prod`	copied into OpenNMS at every start	copied in	lost
OpenNMS's `etc/` (the volume `opennms_data-config`)	an apply, `./topo.sh deploy prod`	kept	kept	lost
your source repository (git)	you, a commit			kept

The image copies three folders into OpenNMS at every start: `/opt/opennms-overlay`, `/opt/opennms-etc-overlay` and `/opt/opennms-jetty-webapps-overlay` (on the host: `.../overlays/overlay`, `.../etc-overlay`, `.../jetty-webapps-overlay`). A file in the

container whose overlay copy is older is replaced by the old one at the next start. That is why the studio and the kit write a changed file into the container **and** into its overlay folder, and why a file only in the container is not safe.

13.2 Before you apply

- [ ] dev shows the change on the real page, in a new tab, and you have looked at every view you care about
- [ ] the studio's History has a snapshot of exactly this version, with a message that says what it is
- [ ] you compared it with the snapshot of the last production apply (History, *Applies*, then *Compare*)
- [ ] the production panel's plan lists only the files you expect, and their diffs are what you changed
- [ ] you know where the undo is: the panel of the module you apply, *Undo the last … apply on prod*
- [ ] for menu names of `dispatcher-servlet.xml`: you know the JSP bar shows them only after production's next restart

13.3 Apply with the studio

1. **Topology** (or **Menu bar**) > panel *Production* > *Apply to production…* > type `production` > confirm.
2. Read the result: each file in the container and its overlay copy (`/opt/maven-nms/data/overlays/jetty-webapps-overlay/opennms/...`), each with an md5 check, and for the style file and your icon copies *OpenNMS serves svg/…*. A mapping file of `etc/` gets an overlay copy only where an overlay folder holds one already; otherwise the volume keeps it.
3. Open production's topology page in a **new tab** (or a private window): `http://192.168.0.36:8980/opennms/topology`. A tab that was open before a new svg file arrived does not have it.

What happened: the commands of a dev apply, through `ssh maven-admin@192.168.0.36 podman ...`, plus the overlay copies, written through the container's mount of the overlay folder. The files replaced went to `backups/` of the studio. Nothing restarted.

The studio sends what it manages: the style file, its block of icon mappings in `icons.default.cfg`, your repaired icon files, and the menu files. Your nimbus nodes get the repaired `nimbus.svg` with this apply.

13.4 Files the studio does not send: the kit

Views, lines in `icons.linkd.cfg`, new icon files and the Change Icon list are the kit's (Chapter 10). They are files in `kit/opennms/`, and one command takes what changed to production:

```
kit$ ./topo.sh diff              # what you changed since the last pull or
production deploy
kit$ ./topo.sh deploy prod       # lists the files, asks, keeps production's
old files in backups/, copies; overlay copies too
```

A file that production changed after your last pull (its software wrote a view again, someone used Change Icon), and you did not, stays as production has it; a file that both changed is held back until you send it with `FORCE=1 ./topo.sh deploy prod`. `./topo.sh pull` makes production's current files your new starting point.

Production's views (*Maven-abc*, *MavenNebulaDAS*) are written by your own software and may be written again at any time. Change their look around the file: icons named after the node types, vertex lines for single nodes, CSS. A view that you change by hand on production can be overwritten by the next run of that software.

13.5 Check, and undo

```
kit$ ./topo.sh persist       # each file you sent: in the container, in the
overlay folder, the same in both
kit$ ./topo.sh css prod      # does production serve your current CSS
kit$ ssh maven-admin@192.168.0.36 podman exec opennms md5sum
/usr/share/opennms/jetty-webapps/opennms/svg/zz-topology-styles.svg
```

Expect: `persist` ends with every file *kept at every start and in a new container*; a line with *NOT SAFE* says which step is missing and what to do (usually `./topo.sh deploy prod` once more). `css prod` says production *has it*.

Undo, by the tool that applied:

Applied with	Undo with	Puts back
the studio	the panel of that module: *Undo the last topology apply on prod* (or menu), type `production`	the replaced files, in the container and the overlay folders; it removes the files the apply added. Each click goes one apply further back
the kit	`./topo.sh undo`	what the last `deploy prod` replaced or added, in the container and the overlay folders; run it again to go one deploy further back

13.6 Before an upgrade of OpenNMS

A new image brings new versions of OpenNMS's own files: `navbar.ftl`, `dispatcher-servlet.xml` and the ui bundles. Your overlay copies of them would replace the new versions at every start. Before the upgrade, remove those copies on production, upgrade, then apply your menu words again (the new bundle has a new name):

```
prod$ O=/opt/maven-nms/data/overlays/jetty-webapps-overlay/opennms
prod$ ls -R $O/WEB-INF $O/ui
prod$ rm $O/WEB-INF/templates/navbar.ftl
prod$ rm $O/WEB-INF/dispatcher-servlet.xml        # only if you renamed menus
prod$ rm $O/ui/assets/index-*.js
```

Your style file and your icons stay: they are yours, not OpenNMS's.

13.7 For good: your source repository

The overlay folders survive restarts and new containers, but not a new machine. Keep the files where your production is defined:

```
kit$ ./topo.sh export        # a folder with your files in the layout of the
overlay folders, and a script that installs them
```

`workspace/` of the studio is a git repository already: push it where your team keeps code. The permanent way for the menu words is a change in the OpenNMS source your image is built from (docs/opennms-menu.md, part 5). kit/GUIDE-production-permanent.md has every manual step, with plain `scp`, `podman cp` and `install`.

The production checklist

- [ ] before: 13.2, all ticked
- [ ] apply: the studio for CSS, mappings, repaired icons and menu words; `./topo.sh deploy prod` for views, Linkd lines and new icon files
- [ ] look: production's topology page in a new tab or a private window
- [ ] check: `./topo.sh persist` and `./topo.sh css prod` say kept and current
- [ ] record: the snapshot of this apply has a clear message; `./topo.sh export` if your source repository holds these files
- [ ] if wrong: the undo of the tool that applied (13.5)

Reference

Appendix A: The file map

Every file this book touches, what it decides, when OpenNMS reads it, and how to change it. Paths in the first table are below OpenNMS's home in the container, `/usr/share/opennms` (`/opt/opennms` is a link to it).

A.1 In the OpenNMS container

File	Decides	OpenNMS reads it	Change it	Chapter
`jetty-webapps/opennms/svg/*.svg`	the icon drawings: groups `X`, `X_rollover`, `X_active`	the list when a browser tab opens the topology page; the content at each load of that tab	`podman exec -i ... cat >`; the studio's repaired copies; `./topo.sh deploy`	8
`jetty-webapps/opennms/svg/zz-topology-styles.svg`	your CSS, inside a `<style>`	like any svg file	the studio's Apply; `./topo.sh deploy`	9, 11
`etc/org.opennms.features.topology.app.icons.default.cfg`	`default`, and lines for single vertices of any view	within a second of a change	a line added; the whole file written to `.new` and renamed; the studio's block; `./topo.sh view icon`	8.3
`etc/org.opennms.features.topology.app.icons.linkd.cfg`	Enhanced Linkd icons, by model	within a second	the same; `./topo.sh deploy`	8.5

File	Decides	OpenNMS reads it	Change it	Chapter
`etc/org.opennms.features.topology.app.icons.pathoutage.cfg`, `.bsm.cfg`, `.application.cfg`, `.vmware.cfg`	the icons of those views	within a second	the same	8.2
`etc/org.opennms.features.topology.app.icons.list`	the icons that right-click > Change Icon offers	when the dialog opens	one id per line	8.5
`etc/org.opennms.features.topology.app.cfg`	settings of the topology app: header, auto refresh, cache	the app reloads itself when it changes	the kit's example 10	FILES.md 4
`etc/graphml/<name>.xml`	a GraphML view: nodes, links, iconKeys, links to OpenNMS nodes	only when the view is loaded	DELETE and POST; or write it, then move the two `.cfg` files out and back; `./topo.sh view add`, `view reload`, `deploy`	7
`etc/org.opennms.features.topology.plugins.topo.graphml-<name>.cfg`	puts the view into the View menu (`topologyLocation`, `label`)	when it appears	written by `POST /rest/graphml/<name>`	7
`etc/org.opennms.netmgt.graph.provider.graphml-<name>.cfg`	the same for the Graph API	when it appears	the same	7
`etc/imports/<name>.xml`	the nodes of a requisition, as last imported	written by OpenNMS at an import	`POST /rest/requisitions`, then an import; `./dev.sh copy-prod`	6
`etc/foreign-sources/<name>.xml`	how the nodes of a requisition are scanned	at an import	`POST /rest/foreignSources`	6
the database	nodes, their ids, sysObjectIDs, alarms	always	REST: `/rest/nodes`, `/rest/events`; imports	6, 8.5
`jetty-webapps/opennms/WEB-INF/templates/navbar.ftl`	the JSP menu bar: Plugins, Help, the user menu	about 5 seconds after a change	the studio's Menu bar	12
`jetty-webapps/opennms/WEB-INF/dispatcher-servlet.xml`	the menu names of both bars	JSP bar: at start; Vue bar: at every request	the studio's Menu bar	12
`jetty-webapps/opennms/ui/assets/index-<hash>.js`	the Vue menu bar, compiled: the word Plugins	by the browser, at each page load	the studio's Menu bar	12
`jetty-webapps/opennms/ui/index.html`	which bundle the Vue pages load	at each page load	read only	12

File	Decides	OpenNMS reads it	Change it	Chapter
`system/.../org.opennms.features.topology.themes.default-theme/<version>/*.jar`	the stock stylesheet of the topology page	at start	never: override it with your CSS; `./topo.sh pull` copies it to `kit/reference/` for reading	9
`logs/karaf.log`, `logs/provisiond.log`, `logs/web.log`	what happened: views, imports, the web UI		read: `tail`, `grep`, `./dev.sh logs`	5

A.2 On the production machine

Folder or name	Holds	Chapter
container `opennms`	the production OpenNMS; next to it `postgres-database` and `maven-nms`	0, 13
`/opt/maven-nms/data/overlays/overlay`	copied below OpenNMS's home at every start (in the container: `/opt/opennms-overlay`)	13.1
`/opt/maven-nms/data/overlays/etc-overlay`	copied into `etc/` at every start (`/opt/opennms-etc-overlay`)	13.1
`/opt/maven-nms/data/overlays/jetty-webapps-overlay`	copied into `jetty-webapps/` at every start (`/opt/opennms-jetty-webapps-overlay`): your style file, icons, menu files	13.1
volume `opennms_data-config`	OpenNMS's `etc/`: kept across restarts and new containers	13.1

A.3 In the project folder

File or folder	Holds	Written by
`workspace/topology/topology.css`	your CSS for the topology page	the studio's editor; any editor

File or folder	Holds	Written by
`workspace/topology/icon-mappings.json`	the studio's vertex lines, `{"<namespace>.<node id>": "<icon id>"}`	the studio's inspector
`workspace/topology/icons/<name>.svg`	your repaired copies of icon files	the studio's *Icon files*
`workspace/menu/labels.json`	your menu words	the studio's Menu bar
`backups/`	the files each studio apply replaced; `journal.jsonl` lists every apply	the studio
`kit/kit.env`	addresses, container names, dev's login, `STYLE_CSS`	you
`kit/icon-types.cfg`	node type = icon id	you
`kit/opennms/`	production's topology files as of the last pull, with your changes	`./topo.sh pull`, you
`kit/.kit/`	production's files at the last pull, copies of production's overlay folders	the kit
`kit/backups/`	what each `./topo.sh deploy prod` replaced	the kit
`kit/reference/topo_default-styles.css`	the stock stylesheet, for reading	`./topo.sh pull`

Appendix B: Commands

For dev; for production put `ssh maven-admin@192.168.0.36` in front of a `podman` command and use the container `opennms`. Change production only as Chapter 13 says.

B.1 Into the container

```
podman ps --format '{{.Names}}  {{.Status}}'                      # the
containers and their state
podman exec -it onms-dev-core bash                                # a shell
inside; exit leaves it
podman exec onms-dev-core ls /opt/opennms/jetty-webapps/opennms/svg
podman exec onms-dev-core sh -c 'ls
/opt/opennms/etc/org.opennms.features.topology.app.icons.*'   # quotes: the *
is expanded inside
podman exec onms-dev-core cat /opt/opennms/etc/<file> > <file>.orig
# read, and keep a copy
podman exec -i onms-dev-core sh -c 'cat > /usr/share/opennms/<path>' < <local
file>             # write a file
F=/usr/share/opennms/etc/<name>.cfg
podman exec -i onms-dev-core sh -c "cat > $F.new && mv $F.new $F" < <local
file>               # write a .cfg safely
podman exec onms-dev-core grep -o 'id="[^"]*"' /opt/opennms/jetty-
webapps/opennms/svg/<file>.svg  # the ids of an svg file
podman exec onms-dev-core tail -n 50 /opt/opennms/logs/karaf.log
# the log of the topology
```

B.2 The REST API

```
U='-u admin:admin'; O=http://localhost:8980/opennms
curl -s $U -H 'Accept: application/json' "$O/rest/nodes?limit=0" | python3 -m
json.tool          # all nodes
curl -s $U -H 'Accept: application/json' "$O/rest/nodes/<foreign source>:
<foreign id>"          # one node
curl -sS $U -X POST -H 'Content-Type: application/xml' --data-binary
@<req>.xml "$O/rest/requisitions"
curl -sS $U -X PUT "$O/rest/requisitions/<name>/import?rescanExisting=false"
# import
curl -sS $U -X DELETE "$O/rest/requisitions/deployed/<name>"
curl -sS $U -X POST -H 'Content-Type: application/xml' --data-binary
@<view>.xml "$O/rest/graphml/<name>"
curl -sS $U -X DELETE "$O/rest/graphml/<name>"
# no PUT: DELETE, wait 5 s, POST
curl -s $U "$O/rest/graphml/<name>"
# XML only: never ask for JSON
curl -sS $U -X PUT -H 'Content-Type: application/x-www-form-urlencoded' --
data 'sysObjectId=<oid>' "$O/rest/nodes/<fs>:<fid>"
curl -s $U "$O/api/v2/menu" | python3 -m json.tool | grep '"name"'
# the menu names of the Vue bar
```

B.3 The kit (in `kit/`)

In this list, `a | b` means one of the words, and `[x]` a word you may leave out.

```
./dev.sh up | status | down | reset | logs [file]       # the development
OpenNMS
./dev.sh copy-prod [requisition...]                       # production's
requisitions (its nodes) and views, into dev
./dev.sh remove-nodes <requisition>                       # a requisition and
its nodes, out of dev
./dev.sh seed                                             # the demo again
./topo.sh pull                                            # production's
topology files into opennms/, and onto dev
./topo.sh diff | revert [file]                            # your changes in
opennms/; drop them
./topo.sh deploy dev | deploy prod | undo                 # copy what
changed; production: asks, backs up, can undo
./topo.sh watch                                           # deploy every save
to dev
./topo.sh icons [dev|prod]                                # icon ids, flawed
files, the icon each node gets
./topo.sh css [dev|prod] | css <icon id>                  # your CSS there,
clashing rules; CSS for the parts of an icon
./topo.sh view list | show <name> | add <file> [name] | remove <name> |
reload [name...]
./topo.sh view icon <name> <node id> <icon id | --reset>
./topo.sh view from-requisition <requisition>             # a view whose
icons follow the node type (icon-types.cfg)
./topo.sh persist | export [folder]                       # does production
keep your files; a folder for your repository
./topo.sh example [N] | pagecheck                         # the examples of
FILES.md; a check to paste into F12 > Console
```

B.4 The studio (in the project folder)

```
npm install            # once, and when an update says so
npm run build          # after every update
npm test               # its unit tests
npm start              # the studio: http://192.168.0.47:8990/?token=...
cat .studio/token      # the access token, if you lost the address
```

B.5 git, in `workspace/`

```
git status; git diff HEAD                              # what changed since the
last snapshot
git add -A && git commit -m "what and why"             # a snapshot
git switch -c experiment/<name>                        # an experiment
git log --oneline --graph --all                        # the history
git restore --source=<snapshot> -- topology menu       # take the files of a
snapshot back
```

Appendix C: When something goes wrong

Find the symptom, read the cause, do the fix. The chapter in the last column explains it.

C.1 The development OpenNMS

Symptom	Cause	Fix	See
`./dev.sh status` says there is no development OpenNMS, or it does not answer	it is stopped, or was never started	`./dev.sh up`	4.2
the start stops with exit code 137	not enough memory	give the VM 4 GB or more	kit README
the log ends with `java: Operation not permitted`	the container may not hold the capabilities the image's java needs	keep `DEV_PODMAN_ARGS` and `DEV_CAP_ADD` of `kit.env` as delivered	kit README
"Access denied" right after a login	the login form went without its CSRF token	reload the login page and log in again; `./dev.sh status` tests a login	kit README
the page is unreachable from your computer	the firewall	`sudo firewall-cmd --permanent --add-port=8980/tcp && sudo firewall-cmd --reload`	4.2

Symptom	Cause	Fix	See
a REST call answers 401	wrong user or password	the password you set in the web UI goes into `kit/kit.env` as `DEV_PASSWORD`	4.3
`ssh` to production asks for a password	no key	`ssh-copy-id maven-admin@192.168.0.36`	0.3
production's nodes or views are missing on dev	not copied yet	`./dev.sh copy-prod`	4.5
a demo node has a stock icon although an svg file has the icon of its type	a kit before 3.12 made the demo	`./dev.sh seed`	4.4

C.2 Views and nodes

Symptom	Cause	Fix	See
a view is not in the View menu	its `.cfg` file is missing, or OpenNMS could not read the xml	`./topo.sh view list`; `karaf.log` names the error; post it again	7.1
a changed view file shows nothing new	OpenNMS reads the xml only when the view is loaded	`./topo.sh view reload <name>`, or DELETE and POST	3.4, 7.3
a node of a view has no status colour	the view has no `vertex-status-provider`; the node is not linked; the view was loaded before the node existed	add the property; link by `foreignSource` and `foreignID`; `./topo.sh view reload`	3.5, 7.4
a node shows the status of another node	it links by `nodeID`, which is another node on this OpenNMS	link by `foreignSource` and `foreignID`	3.5, 6.3

Symptom	Cause	Fix	See
a requisition's nodes do not come	the import was not run, or provisiond is still working	`PUT .../import`; `./dev.sh logs provisiond.log`	6.1
the GraphML REST call answers 406 or 500	it was asked for JSON	no `Accept: application/json` for `/rest/graphml`	B.2

C.3 Icons

Symptom	Cause	Fix	See
a node has no icon, only its frame and label	no svg file defines its icon id	`./topo.sh view show <name>` flags it; correct the iconKey, or add the icon	8.2
a node has the `generic` icon	no vertex line, and its key matched nothing	a vertex line, an iconKey, or a model line	8.2
all nimbus nodes are light blue, and rules for `#nimbus_default` do nothing	`nimbus.svg` stacks its three looks inside `#nimbus`	repair the file: two ids	8.7, 8.8, 11.4
a mapping line for a type works today and not tomorrow	a GraphML view answers for its own iconKeys too, in no fixed order	name the icons after the types; `./topo.sh view from-requisition`	8.6
a vertex line does nothing	the namespace or the node id is wrong	`./topo.sh view show <name>` prints both	8.3
a new icon file does not show	the open tab keeps its list of svg files	open a new tab	2.8, 8.4
right-click > Change Icon removed the comments of a mapping file	OpenNMS writes the whole file anew	expected; keep your copies	8.5

C.4 CSS

Symptom	Cause	Fix	See
a rule changes nothing	a more specific rule wins; or its shapes lie under another look; or an inline style wins	F12 > Styles shows the winner; start with `svg` and name the classes; `!important` only against inline styles	9
a rule changes the colour of the wrong icons	two files define the same class, such as `.cls-1`	start the rule with the drawing's id: `#nimbus .cls-1`; `./topo.sh css` lists such classes	9.2
a change of the CSS does not show after a reload	the browser kept its copy	F12 open, *Network* > *Disable cache*; `./topo.sh css` prints `Cache-Control`	9.1
dev shows the CSS, production does not	production has an older file, or the tab is older than the file	`./topo.sh css prod`; apply to production; a new tab or a private window	13
after a restart, production shows the old look	an older overlay copy came in at the start	`./topo.sh persist`; apply or `./topo.sh deploy prod` again	13.1

C.5 Menu bar and studio

Symptom	Cause	Fix	See
the JSP bar says one word, the Vue bar another	only one of `navbar.ftl` and the compiled bundle was changed	the studio's Menu bar changes both	12.1
a new menu name shows on Vue pages only	the JSP bar reads `dispatcher-servlet.xml` at start	restart OpenNMS	12.3
the studio asks for `?token=`	no token cookie in this browser	open the address `npm start` printed, or use `.studio/token`	11.1

Symptom	Cause	Fix	See
the studio says production is not reachable	ssh needs a password, or `PROD_SSH` is wrong	`ssh-copy-id`; check `kit/kit.env`	11.2
the studio's preview is empty	it could not read the files of that OpenNMS	the message under the preview; *Read the files again*	11.4
`npm install` fails	no internet, or a proxy	`npm config set proxy http://...`	11.1

Appendix D: Glossary

Word	Meaning
alarm	what OpenNMS raises for a problem of a node, with a severity; the worst unacknowledged one colours the node's frame
apply	the studio writing your files into an OpenNMS, with a backup and checks; on production also into the overlay folders
CDATA	`<![CDATA[ ... ]]>` in an XML file: text taken as it stands, so CSS can hold `>` and `&`
category	a label of a node in a requisition, such as `Nimbus`; the kit uses it as the node's type
Change Icon	right-click on a node of the topology page: writes a vertex line into a mapping file
dev	the development OpenNMS, containers `onms-dev-core` and `onms-dev-db` on 192.168.0.47
edge	a link on the map, drawn as `path.edge`
Enhanced Linkd	the view of all nodes with the links OpenNMS found; namespace `nodes`; icons by model
experiment	in the studio, a line of snapshots you can switch to and from: a git branch

Word	Meaning
focus	the vertices a view opens on; with the semantic zoom level it decides what you see
foreign source, foreign id	the requisition of a node and its id there, such as `DAS-Demo:nimbus-2`: the same in every OpenNMS
GraphML	an XML format for graphs; one file in `etc/graphml/` is one view
icon file	an svg file of `jetty-webapps/opennms/svg/` with one or more icons
icon id	the id `X` of the drawing a node shows; OpenNMS also draws `X_rollover` and `X_active`
icon key	what a vertex brings to the icon lookup: a GraphML node's `iconKey` (which is its icon id), or `linkd.system.snmp.<sysObjectID>` in Enhanced Linkd
import	making the database match a requisition: nodes added, changed, deleted
inspector	in the studio, the box that explains the node you clicked in the preview
JSP bar	the menu bar of the JSP pages and the topology map, made by the server from `navbar.ftl`
Karaf	the part of OpenNMS that runs the topology app and reads the `.cfg` files of `etc/` when they change
mapping file	`etc/org.opennms.features.topology.app.icons.<name>.cfg`: lines `<key> = <icon id>`
namespace	the name of a view's graph, such as `das-demo`; with a vertex id it names a vertex
node	a device OpenNMS knows, with a node id, a label, and alarms
node id	the number an OpenNMS gives a node: different on dev and production

Word	Meaning
overlay folder	a folder the image copies into OpenNMS at every start; on production below `/opt/maven-nms/data/overlays/`
Path Outage	the view of all nodes under their parent nodes
provisiond	the part of OpenNMS that imports requisitions and scans nodes
requisition	an XML list of the nodes OpenNMS should have, named by its foreign source
semantic zoom level	how many links away from the focus a view shows
snapshot	in the studio, a saved version of your files: a git commit
specificity	the weight of a CSS selector: ids, then classes, then element types
stacked layout	an icon file whose group `X` holds all three looks, so a node always shows the last one
status	the severity class of a node's frame: `normal`, `warning`, `minor`, `major`, `critical`
svg folder	`jetty-webapps/opennms/svg/`: every file there goes into the topology page
sysObjectID	the model number a device's SNMP agent reports, such as `.1.3.6.1.4.1.99999.1.4`
topology provider	the part of OpenNMS behind one entry of the View menu
vertex	a node on the map, named by namespace and id
vertex line	a mapping line `<namespace>.<vertex id> = <icon id>`: one node, another icon
view	an entry of the View menu: Enhanced Linkd, Path Outage, a GraphML view
Vue bar	the menu bar of the Vue pages, compiled into `ui/assets/index-<hash>.js`

Word	Meaning
workspace	the studio's folder of your files, a git repository
`zz-topology-styles.svg`	the svg file that carries your CSS into the topology page

Appendix E: The other documents of the project

This book is the way in. The documents below go deeper; each is the reference for its part.

Document	Where	Read it when
`README.md`	the project folder	you install, start or update the studio; its commands and its safety rules
`GUIDE.md`	the project folder	you want more studio experiments: T1 to T8 for the topology, M1 to M6 for the menu bar, the production steps
`CHANGES.md`	the project folder	an update arrives: what it changed, and what to do
`docs/opennms-topology.md`	the project folder	you want the topology page from OpenNMS's source: every step, with links to the lines
`docs/opennms-menu.md`	the project folder	you want every word of the menu bar and the file it comes from
`docs/how-the-studio-works.md`	the project folder	you want every command the studio runs, an apply step by step, its API
`kit/README.md`	the kit	you need a kit command, the details of `./dev.sh up`, or the kit's troubleshooting list
`kit/FILES.md`	the kit	you want one topology file explained, with an example to run (`./topo.sh example`)
`kit/GUIDE-node-icons.md`	the kit	you want node icons in depth: the source of Chapter 8

Document	Where	Read it when
`kit/GUIDE-icon-backgrounds.md`	the kit	you want to change the green behind the icons, for all icons or one
`kit/GUIDE-Maven-abc-styles.md`	the kit	you style one production view from start to finish
`kit/GUIDE-production-permanent.md`	the kit	you make files permanent on production by hand, or keep them in a repository
`claude/opennms-ui-studio.md`	your Claude project	you want the state of the project, version by version, and what is still open

When you come back after a pause: read the Checkpoint of the chapter where you stopped, run `kit/dev.sh status`, and go on from there.